High-Throughput Permissionless Blockchain Consensus under Realistic Network Assumptions

Sandro Coretti[1], Matthias Fitzi[2], Aggelos Kiayias[3], Giorgos Panagiotakos[4], and Alexander Russell[5]

[1] IOG, France
sandro.coretti@iohk.io
[2] IOG, Switzerland
matthias.fitzi@iohk.io
[3] University of Edinburgh & IOG, UK
akiayias@inf.ed.ac.uk
[4] IOG, Greece
giorgos.panagiotakos@iohk.io
[5] University of Connecticut & IOG, USA
acr@cse.uconn.edu

Abstract. Throughput, i.e., the amount of payload data processed per unit of time, is a crucial measure of scalability for blockchain consensus mechanisms. This paper revisits the design of secure, high-throughput proof-of-stake (PoS) protocols in the *permissionless* setting. Existing high-throughput protocols are either analyzed using overly simplified network models or are designed for permissioned settings, with the task of adapting them to a permissionless environment while maintaining both scalability and adaptive security (which is essential in permissionless environments) remaining an open question.

Two particular challenges arise when designing high-throughput protocols in a permissionless setting: *message bursts*, where the adversary simultaneously releases a large volume of withheld protocol messages, and—in the PoS setting—*message equivocations*, where the adversary diffuses arbitrarily many versions of a protocol message. It is essential for the security of the ultimately deployed protocol that these issues be captured by the network model.

Therefore, this work first introduces a new, realistic network model based on the operation of real-world gossip networks—the standard means of diffusion in permissionless systems, which may involve many thousands of nodes. The model specifically addresses challenges such as message bursts and PoS equivocations and is also of independent interest.

The second and main contribution of this paper is Leios, a blockchain protocol that transforms any underlying low-throughput base protocol into a blockchain achieving a throughput corresponding to a $(1-\delta)$-fraction of the network capacity—while affecting latency only by a related constant. In particular, if the underlying protocol has constant expected settlement time, this property is retained under the Leios overlay. Combining Leios with any permissionless protocol yields the first near-optimal throughput permissionless "layer-1" blockchain protocol proven secure under realistic network assumptions.

1 Introduction

1.1 Overview

Permissionless blockchain protocols, beginning with Bitcoin [39], allow a large, continuously evolving set of mutually distrusting parties to agree on an ever-growing ledger of transactions as long as the majority of a certain underlying resource—e.g., computation, memory, stake—is controlled by the honest parties.

These protocols are generally leader-based, i.e., new leaders are repeatedly elected to propose new transaction-carrying blocks to be added to the ledger. Leader election is implemented by a resource-based lottery wherein the probability to win is proportional to a party's relative portion of the overall resources dedicated to the system.

Nakamoto vs. BFT style protocols. Towards reaching agreement on the ledger, new blocks must be confirmed—which is achieved either by having (a) future leaders build on them and thereby voting for them implicitly or (b) a committee (also elected via the resource-based lottery) voting for them explicitly. Protocols that follow (a) are generally known as "Nakamoto-style" protocols (cf. [39]), in reference to the author of the original Bitcoin paper, whereas protocols that take approach (b) are referred to as "BFT-style" protocols (e.g., [27]), acknowledging the fact that they employ techniques from Byzantine fault-tolerant computing, a research area that predates blockchain systems by several decades.

Nakamoto-based protocols generally tolerate attackers controlling less than half of the network's resources, be it work or stake, (cf. [39,24,32]) and an uncertain or even fluctuating number of participants, (cf. [25,45,6]) while permissionless BFT-based protocols typically (even though not exclusively) tolerate attackers controlling less than one-third of the resources and operate under predictable participation (e.g., [27]). In terms of settlement, BFT-based protocols offer fast finality, where confirmed blocks are permanently part of the ledger, whereas Nakamoto-based protocols offer a weaker notion of settlement, where the probability that a block remains stable slowly increases over time—at the benefit, however, that they can "self-heal" after short periods of adversarial majority [2,5]. Furthermore, handling uncertain participation as in Nakamoto-style protocols requires a network with bounded message delays (cf. [44]), whereas many BFT-based protocols exploit a level of certainty in participation to function in a partially synchronous network, where delay guarantees only apply after an initial period of uncertain duration (or where delays are limited by an unknown bound), or even in a fully asynchronous network.

The permissionless setting. In a permissionless setting, typically, many thousands of nodes are expected, something that necessitates low communication complexity: Nakamoto consensus achieves optimal communication in the sense that its amortized complexity is just a single message diffusion per block of transactions, which occur at a low rate over time; BFT-based consensus protocols require voting for each block, and thus to reduce the high message overhead, it becomes essential to elect small committees for each voting round. Consequently, in both cases, only a small subset of participants is responsible for the protocol at any given moment. This underscores the vulnerability to low-resource *adaptive* attacks, where an attacker only needs to corrupt a small group of leaders or committee members (representing a minor portion of the total underlying resources) to compromise the protocol.

It follows that adaptive security is crucial for permissionless protocols. Nakamoto consensus is adaptively secure by design since the role of the block leader is limited to creating and diffusing a block, making it "maximally ephemeral." BFT-based protocols (that scale) only achieve adaptive security if the committees rotate frequently, ideally after every message sent (as in, e.g., Algorand [27]).

Challenges for high throughput protocol design. Throughput, i.e., the number of payload data processed per unit of time, is one of the most crucial scaling metrics for blockchains. The goal is to confirm payloads at a rate close to what the underlying network allows. In order to design *adaptively secure* high-throughput permissionless blockchain protocols, several challenges must be met:

- *Consensus is in the way:* In "conventional" blockchain protocols, block production and certification are somewhat sequential: Nakamoto protocols crucially rely on (honest) leaders building on each other's blocks in order to achieve consensus; in many BFT protocols, one or multiple leaders propose new blocks and parties subsequently agree on which block to finalize, which evidently requires proposed blocks to reach sufficiently many honest participants. In both cases, the lag incurred by block propagation for the sake of achieving consensus inevitably sacrifices throughput since the overall network remains underutilized waiting for a single or a few blocks to propagate to all nodes. It is therefore necessary to somehow decouple diffusing payloads and consensus protocol logic in order to achieve high throughput.
- *Message bursts:* Any blockchain protocol must be able to handle the simultaneous release of a large number of *valid* protocol messages (e.g., withheld old blocks) by adversarial parties. This is particularly challenging in the permissionless setting, where nodes are usually connected by gossip networks (NB:

complete point-to-point connectivity is impractical given the large number of participants). In such networks, it is infeasible to throttle messages according to their sources (a common DoS mitigation), and hence, message relaying rules are limited to coarse criteria, such as a resource lottery win and a network-wide freshness condition (e.g., the message points to the hash value of a recent block). The obvious countermeasure to message bursts is to leave sufficient bandwidth unused so that bursts can be absorbed in the worst-case (as a manifestation of this principle one can point to Bitcoin's 10 minute expected delay between blocks); while effective, this countermeasure is completely antithetical to achieving high throughput and as a result a different mitigation approach is needed.

– *Equivocations:* In the permissionless setting, in order to achieve low communication complexity, protocol messages must be accompanied by winning lottery tickets; absent such a mechanism, it would be easy for an attacker to flood a network with invalid messages.
In the proof-of-work (PoW) setting, where lotteries are implemented by solving cryptographic puzzles against a particular message, a winning ticket is tied to that message, and hence, the only possible reuse of a PoW success is a straightforward replay of a previous protocol message. Message replays can be easily contained in a gossiping network (e.g., by keeping a database of hashes for at least as long as a PoW remains fresh).
In a proof-of-stake (PoS) setting, however, where winning tickets are determined cryptographically, e.g., based on the output of a verifiable random function (VRFs), on a purely identity- and time-based input, there is no such tie between a specific lottery win and a message due to the fact that rewinding the lottery winning message computation is (essentially) without cost. Thus, an attacker may create so-called *equivocations*, i.e., *arbitrarily* many legitimate-looking protocol messages backed by a single lottery ticket. Therefore, such equivocations must be dealt with suitably.

1.2 Our Contributions

We present a new blockchain protocol, *Leios*, and analyze it within a new formal model that is suitable for expressing all the salient requirements of high-throughput operation in the permissionless setting. Our results in more detail are as follows.

QUEQ: A more realistic network model. The most popular network model for analyzing consensus protocols is the so-called Δ-*delay model* (and variations thereof, cf. [24,42]). This model makes the assumption that all messages sent by honest parties are delivered within a fixed time bound of Δ. As a result, the model abstracts away any details regarding how messages are propagated and any potential network congestion. Importantly, message delays are assumed *independent* of the protocol execution. It is evident that in the context of high-throughput protocols operating near network capacity, this model is too simplistic and fundamentally unrealistic.

To address this, we introduce a network model that is based on how real-world gossip protocols operate that incorporates the way that QUeueing delays and EQuivocations affect message transmission (hence the model name QUEQ). In more detail, any diffused message is split into a fixed size *small* header and a possibly much *larger* body, with headers being delivered predictably within Δ_{hdr}. Message bodies on the other hand are downloaded separately with their availability being subject to *queueing delays* that take place when multiple message bodies compete for network diffusion. Specifically, a particular message body will incur a delay that is a function of its size, the number of other messages that are in transit, and the *relaying policy* implemented by the network; a relaying policy specifies how relay nodes decide which message body to download and forward when multiple messages are available and compete for diffusion.

To reflect the setting of resource-based permissionless message diffusion each message header has an associated "lottery ID"; honest parties only use each lottery ID once, while malicious parties may use the same lottery ID for arbitrarily many messages (aka equivocations). The QUEQ model explicitly incorporates a simple DoS mitigation: each node locally has a preferred header—the one received first—and will never forward more than two headers per lottery ID, with such a pair constituting a *proof of equivocation (PoE)*, i.e., proof that certain lottery ID was equivocated. Furthermore, nodes will only download the body corresponding

to their preferred header. While this strategy protects against bandwidth loss (since only one message body is downloaded per lottery ID) it may leave the honest parties split in terms of the message bodies they downloaded. The QUEQ networking model leaves the resolution of this problem to the consensus protocol by offering the additional guarantee that PoEs, if they exist, are guaranteed to be delivered in a timely manner network-wide. The new network model is expressed as a UC-functionality (cf. [10]), denoted by $\mathcal{F}_{\mathsf{diff}}$.

Leios: High-throughput permissionless blockchain protocol. Our second and main contribution is Leios, a high-throughput protocol that operates over any (low-throughput) "base protocol" in a black-box manner. Leios achieves a $(1-\delta)$-fraction of optimal throughput for any $\delta > 0$, provided the network capacity is sufficiently large. The analysis of Leios is conducted within the QUEQ model. This makes Leios the first adaptively secure, permissionless "layer-1" protocol with near-optimal throughput operating over a more realistic network model. In particular, Leios is designed to handle queuing delays due to message bursts as well as equivocations effectively.

Bandwidth-driven concurrent block generation. Leios follows the input-endorser / fruitchains approach [24,43,32], where parties participate in a stake-based lottery to produce so-called *input blocks (IBs)*. IBs are disseminated over $\mathcal{F}_{\mathsf{diff}}$, and the overall IB data rate can be set to be arbitrarily close to the network's capacity, enabling near-optimal throughput.

Proofs of IB availability. The IBs are included in the underlying base ledger *by reference*, which serializes the payloads and creates the final ledger. However, this decoupling of payloads from consensus introduces a data availability challenge: on the one hand, the final ledger must not include references to IBs unknown to honest parties, as this could lead to "gaps" that may or may not later be filled by the adversary (by releasing the corresponding IB), resulting in either a liveness or a safety violation (depending on how one interprets the gaps). On the other hand, parties cannot simply disregard unresolved IB references in the base protocol. These references might have been created by honest parties, but due to the high-throughput operation near network capacity, with potential message bursts, it might take some time for the referenced IB to diffuse through the network due to queueing delays.

The solution is to certify the availability of IBs using stake-based voting. A certificate ensures that at least one honest party possesses the referenced IB, guaranteeing that the IB will eventually reach all honest parties. Additionally, the ledger rules of the base protocol are modified to require certified IB references as payloads. This allows parties to accept unresolved IB references in the base protocol, knowing that the certificate ensures their eventual availability.

More efficient proofs of availability. Certifying each IB individually, however, would lead to overhead that scales with the number of IBs. This overhead therefore would consume the same constant fraction of the bandwidth, irrespective of the capacity of the underlying network, which clashes with the goal of achieving a $(1-\delta)$-fraction of the optimal throughput for any $\delta > 0$.

To address this issue, we introduce the concept of *endorser blocks* (EBs), which are also produced through a lottery and each collect multiple IBs (by reference). Data availability certificates are generated for EBs, with an EB becoming certified when all referenced IBs are available. Importantly, the EB rate (and hence the proving rate) need only depend on the security parameter and is therefore independent of the IB rate.

The endorsement process is designed to ensure that at least one *certified* EB covers all honestly generated IBs within a given period. Consequently, the rate of IBs referenced by the underlying ledger matches the proportion of honest parties active in the network.

Freshest-first delivery. To manage message bursts—in the Leios case, e.g., the simultaneous release of many withheld IBs by an adversary—a relaying strategy that prioritizes newer IBs over older ones is used in $\mathcal{F}_{\mathsf{diff}}$.[6] This natural and easy-to-implement relaying policy prevents an adversary from delaying the delivery of honest IBs by flooding the network with older ones.

[6] Note that IBs with timestamps from the future can simply be ignored.

Bounding IB delivery times. The "freshest-first" relaying policy introduces some challenges. First, observe that there must be a clear limit on how far back in time EB references can point; otherwise, establishing protocol latency would seem difficult: if an EB referencing a very old IB is certified and added to the base ledger, it could take an excessive amount of time for that IB to be delivered *to all parties*, as it would be delayed by numerous newer IBs.

Given this limit, two issues remain. First, it must be guaranteed that honest IBs are picked up by honest EBs, which requires establishing bounds on the delivery times of honest IBs. Second, it must be ensured that honest EBs are voted for: If the adversary releases an IB to a party just before it produces an EB, that party would reference the IB. It is therefore crucial to ensure that the certification process only start once the global delivery of said IB can be guaranteed, as otherwise the honestly produced EB may not obtain certification.

These issues are addressed by careful analysis based on the delivery guarantees provided by $\mathcal{F}_{\mathsf{diff}}$. Ultimately, there is a tradeoff between throughput optimality and latency: in essence, the closer the IB data rate is to the network capacity, the longer one needs to wait before including IBs in EBs and before voting on EBs for certification.

Equivocations. Recall that equivocations caused by adversarial IBs can lead to a split network where honest parties prefer different IBs for the same lottery ticket. It must therefore be ensured that such equivocated IBs are not referenced by honest parties. Fortunately, it can be shown that, in the event of a split, $\mathcal{F}_{\mathsf{diff}}$ guarantees that all honest parties receive a proof of equivocation (PoE) within a well-defined time bound—and this fact can be exploited to prevent that honest parties reference equivocated IBs.

Pipelined protocol architecture. Given the number of stages required for IBs, EBs, and their associated votes to propagate through the network and form the necessary certificates, Leios employs a pipelined architecture. This ensures that the concurrent generation of IBs continues uninterrupted, preventing bandwidth from being wasted while votes and endorsements are collected by the protocol participants.

Properties of Leios. In the following, let β be the time it takes an IB to traverse (an empty) network. For ease of presentation we present two variants of Leios: Simple Leios, which is useful to understand the approach and techniques involved, and Full Leios. Simple Leios, for a security parameter L and any $0 < \delta < \Theta(\beta/L)$, achieves

- a throughput arbitrarily close to $(1-\delta)\min(L/\eta, 1)\alpha_{\mathrm{H}}$, where α_{H} denotes the fraction of honest stake and where η is the ledger quality of the base protocol,[7]
- latency worsened by at most $\mathcal{O}(\beta\delta^{-3})$,
- security error $e^{-\Omega(L)}$, and
- fairness, meaning that the fraction of adversarial input blocks making up the final ledger is equal to the fraction of corrupted stake,

against an adaptive adversary corrupting up to less than half of the stake (assuming the base protocol is also secure against such an adversary). Note the factor η in the throughput is undesirable, especially when the underlying ledger is a Nakamoto-style blockchain [21], where η is generally quite large. Further, observe that since the security error is only negligible for superlogarithmic L, the asymptotic behavior above only takes effect for quite small δ. For example, if one is only interested in achieving a throughput of half of the available capacity, Simple Leios still imposes a rather high latency (cf. Section 4.4 for more explanations).

Full Leios does not suffer from the above issues and achieves, for any $0 < \delta < \Theta(\beta/L)$,

- a near-optimal throughput of $(1-\delta)\alpha_{\mathrm{H}}$ relative to network capacity,[8]

[7] Intuitively, a chain quality of η means that there is an honest block among any sequence of blocks that was produced during η time slots.

[8] Over any "short" period of time, this level of throughput is essentially optimal since even without the burden of consensus one may at best hope to reach a fraction α_{H} of network utilization, since the adversarial nodes can always "waste" the remaining bandwidth.

– latency worsened by at most $\mathcal{O}(\beta\delta^{-2})$ *in expectation* compared to the base protocol,
– negligible security error independent of L, and
– fairness, as above,

against an adaptive adversary corrupting up to less than half of the stake (assuming the base protocol is also secure against such an adversary). The key insight is that one can circumvent the impact of possibly low underlying ledger quality by having pipelines cross-reference each other. Note further that since the security error is independent of L, L can be chosen (mostly) freely, and the condition $\delta < \Theta(\beta/L)$ does not prevent a proper throughput/latency tradeoff, i.e., one that does not only work for small δ.

For convenience, both variants of the protocol are first presented assuming the adversary does not create any equivocations. We then demonstrate how we can exploit the structure of Leios and proofs of equivocation to handle equivocations and ensure they do not impact the protocol.

In our presentation we focus on the PoS setting, but we also briefly explain in Appendix F, how the construction can be extended to other resources such as PoW. Furthermore, if the base protocol supports dynamic participation (as, e.g., Nakamoto-style protocols generally do), both Simple and Full Leios remain live in the face of fluctuating participation. During times of low participation, the throughput of Leios gracefully falls back to that of the base protocol, without affecting safety or liveness. As participation improves, the high-throughput properties of Leios take effect anew. In this way, for instance, combining our scheme with Ouroboros [32,16] one can obtain an optimal throughput PoS protocol that supports dynamic participation.

1.3 Related Work

Modeling networking for blockchain protocols. Previous analyses of blockchain feature a number of increasingly more realistic network models. Early work was based on so-called "message passing rounds" [24] or the aforementioned Δ-delay model (e.g., [16]).

Bagaria *et al.* [7] refine the Δ-delay model by setting $\Delta := |m|/C + D$ to diffuse a message m, where C is the capacity of the network (in bits per second) and D is a propagation delay (at minimum equal to what is imposed by the speed of light). The protocol they analyze in this model (see below) proceeds in rounds of length Δ and, in order not to overwhelm the network, keeps the data volume to be processed per round below ΔC. The limitation of this approach is that it accounts for neither message bursts nor equivocations.[9]

Neu *et al.* [41] were the first to introduce a model that separates messages into headers and bodies. To diffuse a message, a party first uploads it into a hypothetical *cloud*. A message header diffuses with a fixed delay Δ_{hdr}, and any message body may be downloaded once its header is received, but with a rate bounded by the capacity C. While this model makes an important first step towards recognizing the issue of message bursts (as parties now must explicitly choose which message bodies they download), it still does not account for equivocations: for example, equivocations can happen on the header level, but the model assumes that all headers become available within Δ_{hdr}, irrespective of how many equivocations the attacker attempts to diffuse. Furthermore, in terms of message bursts, the immediate availability of block bodies for download (once the header has been received) does not realistically model diffusion over gossip networks, where bodies only become available after they spend some time traversing the network wherein they may encounter other messages that can delay them further, depending on the relaying policy of the network.

Scaling Nakamoto consensus. A classic approach to overcome the consensus protocol overhead is to decouple consensus blocks from the dissemination of payload data. In the context of blockchain protocols this was highlighted first by Bitcoin-NG [20], where each consensus-block producer continues to append new ledger-transaction blocks (without computing new PoWs) until the next consensus-block gets published. However, this protocol only provides good throughput with game-theoretic guarantees, but not under Byzantine corruption, as a malicious miner can append his consensus block immediately to its previous consensus block and thus censor all transaction blocks the previous miner has appended.

[9] The latter (equivocations) is not needed in [7], as the protocol is PoW-based.

The input-endorsers/fruitchains [24,43,32] technique was originally devised to achieve fairness in Nakamoto-style consensus (i.e., the fraction of blocks a party is able to contribute to the settled blockchain is proportional to their resources), which is provably impossible in pure Nakamoto consensus [21].

Bagaria *et al.* [7] insightfully observed that the input-endorsers technique can be effectively applied towards achieving high throughput by keeping the underlying Nakamoto protocol "minimal" (and safe) but pushing the input-block production towards the bandwidth limit. Their resulting *Prism* protocol, based on PoW, achieves optimal throughput in the limited network model mentioned above. Note that a similar outcome was achieved in a more involved way by Fitzi *et al.* [22] and Yu *et al.* [51] by running multiple quasi-independent blockchain protocols in parallel and merging the multiple chains into a single ledger. This approach is also known as *Parallel Chains*.

A different approach to decoupling consensus blocks from the payload data was given by Zhang et al. [53] where blocks pre-announce transactions to be included in later consensus blocks at a certain minimal distance—to allow for the respective transactions to spread to all mempools before effective inclusion. Ideally, this guarantees that upon retrieval of a block, all included transactions are already in the local mempool, thus minimizing the amount of data to be downloaded at the time a given block is published. Note that, without additional measures, this method cannot provide optimal throughput: it is only effective for the payloads of honest blocks while, in general, the payloads of dishonest blocks still need to be downloaded at the time the block is published (as they may have been pre-announced as required, but not pre-distributed).

Equivocations. Neu *et al.* [41] highlighted first the issue of equivocations in PoS protocols. From our perspective it is worth highlighting that all mitigations offered in [41] impact throughput (or security) as an equivocation adversary may waste honest parties' bandwidth by leading them into pursuing protocol histories that are eventually abandoned or invalid. This problem was identified by Kiffer *et al.* [33] who proposed an equivocation defense that has parties download the bodies of at most one version of each block while achieving longest-chain consensus purely on the chain of headers. This leads to situations in which some parties have downloaded a block that ultimately remains in the longest chain while others have not. They address this via equivocation proofs, which, when included in a later block (before some deadline), result in the contents from the corresponding block being annulled (i.e., they are ignored in the ledger). While equivocation proofs rectify PoS blockchain security, the resulting throughput offered is sub-optimal, and further, the approach impacts latency as well.

Scaling BFT consensus. Classical PBFT [11] suffers from poor throughput since, even in its steady state, a new block is proposed only once during multiple protocol rounds. Hotstuff [49] improved over PBFT by a pipelining construction to allow for a new block to be proposed during every single round. Still, throughput of Hotstuff is suboptimal since the "passive" replicas' upload links are not utilized during each leader's round. To remove this asymmetry, Schiper et al. [38] proposed a PBFT protocol where multiple leaders propose blocks concurrently, with respective follow-up proposals given in [48,28,3].

Another line of high-throughput protocols was initiated by Keidar *et al.* [30], where, in each round, all parties extend a block DAG, and a superimposed sequential leader-election process is applied to eventually agree on a serialization of the DAG blocks. Variants thereof were given in [14,47,31].

Interestingly, all of the above "parallelized" protocol variants can be seen as analogs of input endorsing for throughput as in [7], or vice versa: blocks are proposed concurrently, and eventually ordered by a slim consensus mechanism.

From our perspective it should be emphasized that the above protocols are designed for the *permissioned* setting and would have to be adapted suitably to work in a permissionless environment while retaining adaptive security, which appears non-trivial. A particular approach could be to prove that the protocols satisfy the notion of *player replaceability* [27] and then use sortition to deploy them in a permissionless setting. However, this would also require establishing that the protocols are able to handle message bursts and equivocations—essentially, one needs to show that they remain secure in the QUEQ model, and if they are not, determine whether they can be suitably adapted.

Other work. Further details on previous work, and how our tooling relates to it, are given in supplementary material Appendix A.

Organization of the Paper. Section 2 succinctly presents a model for capturing the security of ledger protocols, including: the protocol interface, security definitions, and the hybrid functionalities that we are going to use. The QUEQ network model is described in detail in Section 3. Sections 4 and 5 are dedicated to the description and analysis of Simple and Full Leios, respectively. Handling equivocations is explained in Section 6.

2 Preliminaries

2.1 Security Definitions

Model. To capture the security of ledger protocols, we adopt a model providing parties access to a number of hybrid functionalities $\mathcal{F}_i$ reflecting infrastructural assumptions such as a (low-throughput) network model, VRFs, signatures, etc. A ledger protocol Π has access to these hybrids and provides the interfaces specified below to the environment.

This work considers only *synchronous* protocols in which parties have access to a shared clock that proceeds in *slots*. For simplicity, only PoS protocols are considered here, but the definitions generalize to other protocols based on other resources. It is assumed that there is a *key registration functionality* (not made explicit) that parties can use to distribute keys and initial stake distribution consistently among each other, as in [16].

Protocol interface. As our overlay protocol will ultimately rely on the base protocol to settle protocol metadata rather than low-level transactions, we keep the transactions abstract and refer to them as *payloads*. In contrast, the payloads of the overlay protocol itself are conventional ledger transactions.

The interface between the environment and a ledger protocol is as follows:

- Initially, the protocol produces an output $(\textsc{Stake}, \mathsf{SD})$ containing the *stake distribution* $\mathsf{SD} = ((P, s_P))_P$, where $s_P \in [0,1]$ and $\sum_P s_P = 1$. Subsequently it takes an input $(\textsc{Init}, V)$, where V is a validation predicate taking a vector of payloads p as input.
- The protocol takes input $(\textsc{FtchLdg})$ and then returns $(\textsc{FtchLdg}, \mathcal{L})$, where $\mathcal{L}$ is a vector of payloads representing the *settled* part of the ledger.
- At any point the environment accepts messages $(\textsc{FtchPld})$ from parties running the protocol and replies to these messages immediately with a message $(\textsc{FtchPld}, p)$, where p is a payload.[10]

The idea is that $(\textsc{FtchPld})$ is *only* called by the party during a slot wherein it produces a block (to determine the payload for the block)—and not during every slot as other liveness definitions suggest. This models the idea that in practice a protocol would fetch transactions from a local mempool when needed (i.e., the mempool is modeled as part of the environment).

Security. The security of the ledger protocol is captured w.r.t. an attacker that may corrupt an arbitrary number of parties as long as the stake corresponding to corrupted parties P—based on the stake distribution SD output initially—remains below a certain threshold (typically 1/3 or 1/2).[11] Throughout this paper, α_{H} denotes the fraction of honest stake.

[10] For simplicity, we restrict the environment to only input p that satisfy $V(p)$ and to input the same V to all parties. It is also assumed that each $\textsc{FtchPld}$ request by the protocol is answered with a payload that fits into an input block. Furthermore, note that the protocol can ask for sufficient amounts of payload to fill a block by repeated use of $\textsc{FtchPld}$.

[11] For simplicity it is tacitly assumed that the protocols considered in this work are well-formed in that they output the same stake distribution to all parties. In particular, it is assumed that protocols have a way for parties to coordinate the initial stake distribution as well as coordinate the key registration process, e.g., as done in [16].

Ledger protocols are expected to satisfy the usual two fundamental properties: Persistence and Liveness, where Persistence requires that all parties have consistent views of the ever-growing ledger and Liveness requires that honest payloads make it into the ledger. Furthermore, the ledgers output by the protocols have to satisfy a validity notion requiring that the settled ledger satisfies V.

Towards the definitions, let $\mathcal{L}_P^{(t)}$ be the random variable corresponding to the settled ledger (output via the FtchLdg command) as seen by party P at time t, and let $\boldsymbol{p}_\tau$ be the set of payloads fetched from the environment by honest parties in slot τ via FtchPld.

The desired properties are captured based on the events defined in Definitions 1-4 below.

Definition 1 (Ledger validity). *For all slots t and all parties P, $V\big(\mathcal{L}_P^{(t)}\big)$ is satisfied.*

Definition 2 (Persistence with parameter w). *For all honest parties P, P' and all times $t \leq t'$, it holds that (i) $\mathcal{L}_P^{(t)}$ is a prefix of $\mathcal{L}_P^{(t')}$ and (ii) $\mathcal{L}_{P'}^{(t-w)}$ is a prefix of $\mathcal{L}_P^{(t)}$.*

Definition 3 (Liveness with parameters r and u). *For all times t, the following holds: if some payload p is included in the responds to all FtchPld request made by every honest party during r rounds, i.e., in each round $\tau = t, t+1 \ldots, t+(r-1)$, $p \in \boldsymbol{p}_{P,\tau}$ for all honest P, then p will be contained in $\mathcal{L}_P^{(t+u)}$ for each honest party P.*

We also rely on the following notion of *ledger quality* [42].

Definition 4 (Ledger Quality with parameters η and u). *For all slots t, the following holds: at least one payload from $\boldsymbol{p}_t \cup \boldsymbol{p}_{t+1} \ldots \cup \boldsymbol{p}_{t+\eta}$ is in the ledgers of all honest parties at time $t+u$, i.e.,*

$$\exists p \in \bigcup_{i=0}^{\eta} \boldsymbol{p}_{t+i} : p \in \mathcal{L}_P^{(t+u)} \textit{for all honest parties } P.$$

Throughput. The throughput of a ledger protocol is defined as the average growth rate of the *sanitized* honest ledger. The sanitized honest ledger at time t, denoted $\mathcal{L}_{\mathsf{san}}^{(t)}$, is the common prefix of all honest settled ledgers $\mathcal{L}_P^{(t)}$ without (a) duplicate payloads (i.e., keeping only, say, the first copy of each payload) and without (b) payloads that were not input by the environment via FtchPld.

Moreover, the throughput of the ledger protocol is defined w.r.t. an environment that inputs (globally) unique payloads whenever requested by the protocol. This is motivated by the fact that when a (reasonably implemented) practical system is under high load, the mempools commonly contain unique transactions. Environments of this type are referred to as *unique-payload environments*.

Definition 5. *A ledger protocol has $(\varepsilon(t), R)$-*throughput *if for every $\gamma > 0$, there exists a time $t_0 \in \mathbb{N}$ such that in an execution with any unique-payload environment,*

$$\Pr\Big[|\mathcal{L}_{\mathsf{san}}^{(t)}|/t \;<\; (1-\gamma)R\Big] \;\leq\; \varepsilon(t)$$

for all $t \geq t_0$.

2.2 Verifiable Random Functions and Lotteries

A *verifiable random function (VRF)*, defined by Micali *et al.* [37], is a cryptographic primitive that allows a party P to create a key pair consisting of a secret evaluation key and a public verification key such that: (a) the public key implicitly defines a function f (b) with the secret key, P can evaluate f at any input x, obtaining an output y and a proof π; (c) given the public key of P, anyone is able to verify, using π, that y is indeed the output corresponding to x; (d) with only the public verification key, outputs of the function cannot be predicted, and in fact appear random.

The above guarantees are abstracted by an idealized functionality $\mathcal{F}_{\mathsf{vrf}}$ (cf. Figure 1). The main commands offered by $\mathcal{F}_{\mathsf{vrf}}$ are (i) (GetKey), answered by (GetKey, v) for an (adversarially chosen) idealized public key v, (ii) (Eval, v, x), answered by (Eval, y, π), and (iii) (Verify, v, x, y, π), answered by (Verify, ϕ) with $\phi \in \{0,1\}$ indicating whether π is a valid proof for the input/output pair (x, y) under public key v. For a realization of $\mathcal{F}_{\mathsf{vrf}}$, consult [16].

Functionality $\mathcal{F}_{\mathsf{vrf}}$

Variables: The functionality keeps track of the following variables, initialized to the values below:

1. an array $\mathsf{Keys}[P] := \emptyset$: set of keys owned by P;
2. an array $T[v, x] := \perp$, where v is a key and x a domain value: pair (y, S), where y is a range value and S a set of proofs π;
3. a set $E := \emptyset$: contains triples (v, x, y) to keep track of all VRF evaluations.

Keys: Upon (GETKEY) from P: Output (GETKEY, P) to $\mathcal{S}$ and ask $\mathcal{S}$ for a unique key v. Add v to $\mathsf{Keys}[P]$ and output (GETKEY, v) to P.

Register Keys: Upon (REGKEY, v) from $\mathcal{S}$: if v is unique, add it to $\mathsf{Keys}[\mathcal{S}]$.

Evaluation: Upon (EVAL, v, x) from P with $v \in \mathsf{Keys}[P]$: Output (EVAL, P, v, x) to $\mathcal{S}$, and upon obtaining (EVAL, π) from $\mathcal{S}$:

1. If π is not unique, exit the procedure.
2. If $T[v, x]$ is undefined, pick y uniformly at random from the range and set $S := \emptyset$; otherwise, let $(y, S) := T[v, x]$.
3. Set $T[v, x] := (y, S \cup \{\pi\})$ and add (v, x, y) to E.
4. Output (EVAL, y, π) to P.

Malicious evaluation: Upon (EVAL, v, x) from $\mathcal{S}$:

- *Case 1:* There exists an *uncorrupted* P with $v \in \mathsf{Keys}[P]$: If $(y, S) := T[v, x]$ is defined, return (EVAL, y) to $\mathcal{S}$; otherwise, do nothing.
- *Case 2:* There exists a *corrupted* P with $v \in \mathsf{Keys}[P]$ *or* $v \in \mathsf{Keys}[\mathcal{S}]$: If $T[v, x]$ is not defined, pick y uniformly at random from the range, let $S := \emptyset$ and set $T[v, x] := (y, S)$; otherwise, let $(y, S) := T[v, x]$. Return (EVAL, y) to $\mathcal{S}$.
- *Else:* Do nothing.

Verification: Upon (VERIFY, v, x, y, π) from any ITI: Send (VERIFY, v, x, y, π) to $\mathcal{S}$, and upon receiving (VERIFY, ϕ) from $\mathcal{S}$:

- If $v \in \mathsf{Keys}[\cdot]$ and $T[v, x] = (y, S)$ is defined:
  1. If $\pi \in S$, set $f := 1$.
  2. Else, if $\phi = 1$ and π is unique—i.e., if for all $(v', x') \neq (v, x)$ with $T[v', x'] = (\cdot, S)$, $\pi \notin S$—set $T[v, x'] := (y, S \cup \{\pi\})$ and $f := 1$.
  3. Else, set $f := 0$.
- Else, set $f := 0$.

Output (VERIFY, f) to P.

Adversarial leakage: Upon (LEAK) from $\mathcal{S}$: return (LEAK, E) to $\mathcal{S}$.

Fig. 1. VRF functionality $\mathcal{F}_{\mathsf{vrf}}$.

2.3 (Stake-Based) Voting

Leios requires a voting scheme that satisfies the following two conditions: (1) if all honest parties vote for a particular message, they are able to generate a certificate for that message; (2) if no honest party votes for a particular message, the adversary is unable to create such a certificate.

These guarantees are captured by a functionality $\mathcal{F}_{\mathsf{sbv}}$ (described next), which is realized by a protocol π_{sbv} via stake-based voting, relying on a verifiable random function (VRF) and a key-evolving signature scheme (KES, cf. Appendix C). The protocol π_{sbv} is secure against an adaptive adversary corrupting less than half of the total stake. The description of π_{sbv} and a security proof are provided in Appendix D.

The Voting functionality. The intuition behind the SBV functionality $\mathcal{F}_{\mathsf{sbv}}$ (cf. Figure 2) is based on the standard way of modeling UC-secure signatures, where public keys and signatures are "symbolic" strings chosen by the adversary, and unforgeability is enforced based on what was signed by uncorrupted parties.

Functionality $\mathcal{F}_{\mathsf{sbv}}$ proceeds in periods, where parties can only sign (vectors of) messages under strictly increasing period numbers. This is due to the realization based on KESs, required to obtain adaptive security (see Appendix C). To that end $\mathcal{F}_{\mathsf{sbv}}$ keeps counters ctr_P that remember, on a per-party basis, the latest period for which a message was signed.

The functionality keeps track of

- public keys using a set Keys of pairs (P, k), meaning that public key k belongs to party P;
- votes using a set Votes consisting of tuples (k, j, m, v, h, f), meaning that v is a period-j vote for m under public key k; $f \in \{0, 1\}$ records whether v is valid and h whether the voter was honest at the time of creation; and
- certificates using a set Certs consisting of tuples $(\mathbf{k}, j, m, c, f)$, meaning that c is a period-j certificate for message m created using votes under keys $\mathbf{k}$; $f \in \{0, 1\}$ records whether c is valid.

Key generation. A party requests a public key from $\mathcal{F}_{\mathsf{sbv}}$ using the GEN command. $\mathcal{F}_{\mathsf{sbv}}$ then asks the adversary to provide a symbolic key string that is different from any keys appearing in sets Keys, Votes, and Certs.

Functionality $\mathcal{F}_{\mathsf{sbv}}$

Variables: The functionality keeps track of the following variables, initialized to the values below:

1. $\mathsf{SD}^* := \bot$: stake distribution used.
2. $\mathsf{Keys} := \emptyset$: a set of tuples (P, k), where P is a party and k is a public key.
3. $\mathsf{Votes} := \emptyset$: a set of tuples (k, j, m, v, h, f) where k is a public key, j is a period number, m is a message, v is a vote, and h and f are bits.
4. $\mathsf{Certs} := \emptyset$: a set of tuples $(\mathbf{k}, j, m, c, f)$ where k is a public key, j is a period number, m is a message, v is a vote, and f is a bit.
5. A period counter $\mathsf{ctr}_P := 0$ for every party P.

Key generation: Upon (Gen) from P: Pass (Gen, P) to the adversary and obtain a value k such that k does not appear in Keys, Votes, or Certs. Add (P, k) to keys and pass (Gen, v) to P.

Stake distribution: Upon (SetSD, SD) from P: If SD is invalid, exit. If $\mathsf{SD}^* = \bot$, set $\mathsf{SD}^* := \mathsf{SD}$. If $\mathsf{SD} \neq \mathsf{SD}^*$, explode.

Voting: Upon (Vote, $k, j, \mathbf{m}$) from P:

1. If $(P, k) \notin \mathsf{Keys}$ or $j \leq \mathsf{ctr}_P$, exit.
2. Pass (Vote, $P, k, j, \mathbf{m}$) to the adversary and obtain a value $\mathbf{v}$ such that
   (a) if $\mathbf{v} \neq \bot$, then $(k, j, \mathbf{m}[i], \mathbf{v}[i], \cdot, 0) \notin \mathsf{Votes}$ for all i and
   (b) if all honest parties except P have called Vote for period j and only $\bot$ have been returned by the adversary so far, then $\mathbf{v} \neq \bot$.
3. If $\mathbf{v} \neq \bot$, add $(k, j, \mathbf{m}[i], \mathbf{v}[i], 1, 1)$ to Votes.
4. Set $\mathsf{ctr}_P := j$.
5. Pass (Vote, $\mathbf{v}$) to P.

Vote verification: Upon (VfyVote, k, j, m, v) from P:

1. Pass (VfyVote, k, j, m, v) to the adversary and obtain a bit ϕ.
2. Determine f as follows:
   – If $(k, j, m, v, \cdot, f') \in \mathsf{Votes}$ for some f', set $f := f'$.
   – Else if $(P, k) \in \mathsf{Keys}$ and P is uncorrupted, set $f := 0$.
   – Else, set $f := \phi$ and add $(k, j, m, v, 0, \phi)$ to Votes.
3. Pass (VfyVote, f) to P.

Certificate generation: Upon (GenCert, $\mathbf{k}, j, m, \mathbf{v}$) from P:

1. Run the verification in VfyVote with $(\mathbf{k}[i], j, m, \mathbf{v}[i])$ for all i; if not all of them succeed, exit.
2. Pass (GenCert, $\mathbf{k}, j, m, \mathbf{v}$) to the adversary and obtain a value c such that
   (a) if $c \neq \bot$, then $(\mathbf{k}, j, m, c, 0) \notin \mathsf{Certs}$ and
   (b) if $\mathbf{k}$ includes all keys k for which $(k, j, m, v, 1, 1) \in \mathsf{Votes}$ and all honest parties have voted for m period j, then $c \neq \bot$.
3. If $c \neq \bot$, add $(\mathbf{k}, j, m, c, 1)$ to Certs.
4. Pass (GenCert, c) to P.

Certificate verification: Upon (VfyCert, j, m, c) from P:

1. Pass (VfyCert, j, m, c) to the adversary and obtain a bit ϕ.
2. Determine f as follows:
   – If $(j, m, c, f') \in \mathsf{Certs}$ for some f', set $f := f'$.
   – Else if $(\cdot, j, m, \cdot, 1, 1) \notin \mathsf{Votes}$, set $f := 0$.
   – Else, set $f := \phi$ and add (k, j, m, c, ϕ) to Votes.
3. Pass (VfyCert, f) to P.

Fig. 2. Stake-based voting functionality $\mathcal{F}_{\mathsf{sbv}}$.

Stake distribution. In order to realize $\mathcal{F}_{\mathsf{sbv}}$ using stake-based voting, after generating their keys, parties must provide a valid stake distribution $\mathsf{SD} = \{(P, s_P, k_P)\}_P$ to $\mathcal{F}_{\mathsf{sbv}}$, using command SetSD. SD is valid if it satisfies $\sum_P s_P = 1$ and k_P is a key belonging to P for all P. Crucially, parties must reach agreement on the stake distribution externally, as the functionality only works if all honest parties input the same SD (otherwise, $\mathcal{F}_{\mathsf{sbv}}$ "explodes," which means the adversary is given full control over it).

Vote generation and verification. Using command Vote, parties may obtain individual votes for arbitrary messages in a vector $\mathbf{m}$ for a particular period, i.e., all votes for a given period must be requested simultaneously. $\mathcal{F}_{\mathsf{sbv}}$ then requests a vector $\mathbf{v}$ of symbolic vote strings from the adversary, who may choose to also reply with the special symbol $\bot$. This flexibility allows to realize $\mathcal{F}_{\mathsf{sbv}}$ via sortition. However, in order to be useful, the functionality will enforce that at least one honest party is given a non-$\bot$ vote in each period, which will be sufficient for $\mathcal{F}_{\mathsf{sbv}}$ to allow the generation of a certificate (see below). $\mathcal{F}_{\mathsf{sbv}}$ ensures that if non-$\bot$ vote strings *are* returned by the adversary, they do not already appear in Votes as invalid.

Vote verification is akin to standard signature verification in UC, where the adversary is asked for a bit ϕ, determining vote validity, unless

– the vote to be verified is already known to be valid or invalid, in which case the answer (the f-bit) in Votes is used, or

– the key under which the signature is to be verified belongs to an honest party, in which case the answer is always 0 (in order to ensure certificate unforgeability).

Certificate generation and verification. Parties may request certificates for messages m by providing a set of corresponding votes under particular keys. The adversary is again asked to provide a symbolic certificate string. It may also return $\bot$, except when all honest parties have voted for m in the period in question and

the set of votes includes all honestly generated votes for m. $\mathcal{F}_{\mathsf{sbv}}$ ensures that if a non-$\perp$ certificate *is* returned by the adversary, it does not already appear in Certs as invalid.

Certificate verification follows the same pattern as above, where the adversary is asked for a bit ϕ, determining certificate validity, unless

- the certificate to be verified is already known to be valid or invalid, in which case the answer (the f-bit) in Certs is used, or
- no honest party has voted for the message in question in the period in question, in which case the answer is always 0 (in order to ensure vote unforgeability).

3 QUEQ: A More Realistic Network Model

An important contribution of this work is the QUEQ model, a new, more realistic network model for analyzing high-throughput consensus for permissionless blockchains (cf. Section 1.2). The model is based on how real-world gossip protocols operate and captures the guarantees they provide.

Therefore, first, an overview of gossip-protocol design is provided below, including a sketch of how one can safely implement high-throughput gossip against Byzantine adversaries. The sketched protocol serves as a justification for the QUEQ model, the most realistic model used to analyze blockchain protocols to date, accounting for QUeueing delays and so-called EQuivocations. The QUEQ model is described as a UC functionality $\mathcal{F}_{\mathsf{diff}}$.

Real-world gossip protocols. Gossip protocols commonly use an expander-like overlay network with low degree and short diameter. The main idea is for new messages to spread by having parties forward them to their neighbors in the network. In order to avoid receiving messages multiple times from different neighbors (which wastes bandwidth), messages are split into small headers and (potentially much) larger bodies; a node, given a header, then decides from which neighbor to download the corresponding body. Note that when aiming to maximize throughput, it is crucial that each body only be downloaded once.

Additional challenges arise in an adversarial environment: to avoid straightforward DoS attacks in which adversarial nodes simply flood the network with (useless) messages, it is important that (i) the application layer (e.g., consensus) restrict the rate of valid messages sent, and (ii) the network layer use the application layer to verify the validity of messages received and drop illicit ones. This is typically accomplished by implementing the lottery using a proof-of-X mechanism, where X can be stake, memory, work, etc.

However, when using PoX naively, the adversary is still able to attack the network by buffering legitimate messages and releasing them in bulk at some point in the future, an issue referred to as *message bursts* in this work. Furthermore, when using proofs of *stake*, which are adopted in this work, the adversary is able to *equivocate* messages, i.e., to send arbitrarily many different messages for the same lottery ticket. Thus, a DoS-safe network layer has to suitably manage equivocated messages.

The concrete gossip protocol after which $\mathcal{F}_{\mathsf{diff}}$ is modeled works as follows: Messages are split into headers h and bodies b with the property that there is a public function $\mathsf{match}(h, b)$ that tests whether a particular b matches a particular header h; furthermore, it is assumed that it is infeasible to find two bodies that match the same header and that a lottery ID $\mathsf{LID}(h)$ is inferable from the header.[12] Lottery IDs are assumed to be from some totally ordered set and can thus also act as priorities.[13]

Header diffusion works as follows:

1. The diffusion of a message is initiated by the sender by first sending its header to all of its neighbors.
2. When a party receives a new header, it is not directly forwarded but rather output to the environment (which corresponds to the application layer).
3. When a party receives a header as input from the environment, they forward it to all of their neighbors.

[12] An equivocation is thus a pair of different (valid) messages (h, b) and (h', b') with $\mathsf{LID}(h) = \mathsf{LID}(h')$.

[13] Note that one can easily generalize this to use priorities that are an arbitrary function of the entire header.

Functionality $\mathcal{F}_{\mathsf{diff}}$

Parameters & Notation: $\Delta_{\mathsf{hdr}} \in \mathbb{N}$, header propagation delay; β: base delay; B: body size; C, capacity of the network; $\mathsf{match}(h, b)$, true iff body b matches header h; $\mathsf{LID}(h)$, lottery ID of h. $\mathbb{T}$ denotes the current time.

Admin

Variables: The functionality keeps track of the following variables, initialized to the values below:

- $\mathsf{Hdrs} := \emptyset$: contains a triple $(h, t, \mathcal{P})$ for each header h diffused, where t is the time of diffusion and $\mathcal{P}$ the set of parties who have already received it.
- $\mathsf{prefHdr}_P[\mathsf{lid}] := \perp$: preferred header of P for message with lottery ID lid.
- $\mathsf{Bdys} := \emptyset$: contains a tuple $(h, b, t, \mathcal{P})$ for header h and b diffused, where t is the time of diffusion and $\mathcal{P}$ the set of parties who have already received it.

Helpers: The description uses the following helpers:

- $\mathsf{hasHdr}(P, \mathsf{lid})$: returns true iff there exists $(h, \cdot, \mathcal{P}) \in \mathsf{Hdrs}$ with $\mathsf{LID}(h) = \mathsf{lid}$ and $P \in \mathcal{P}$.
- $\mathsf{hasPoE}(P, \mathsf{lid})$: returns true iff there exist $(h, \cdot, \mathcal{P}), (h', \cdot, \mathcal{P}') \in \mathsf{Hdrs}$ with $\mathsf{LID}(h) = \mathsf{LID}(h') = \mathsf{lid}$ and $P \in \mathcal{P} \cap \mathcal{P}'$.
- $\mathsf{hasBdy}(P, \mathsf{lid})$: returns true iff there exists $(h, \cdot, \cdot, \mathcal{P}) \in \mathsf{Bdys}$ with $\mathsf{LID}(h) = \mathsf{lid}$ and $P \in \mathcal{P}$.
- $\mathsf{HdrsAdd}(h, t, P)$: if there exists $(h, \cdot, \mathcal{P}) \in \mathsf{Hdrs}$, add P to $\mathcal{P}$; otherwise, add $(h, t, \{P\})$ to Hdrs.
- $\mathsf{BdysAdd}(h, b, t, P)$: if there exists $(h, \cdot, \cdot, \mathcal{P}) \in \mathsf{Hdrs}$, add P to $\mathcal{P}$; otherwise, add $(h, b, t, \{P\})$ to Bdys.
- $\mathsf{higherBdys}(h)$: returns the number lid for which there exists $(h', \cdot, \cdot, \cdot) \in \mathsf{Bdys}$ with $\mathsf{LID}(h') > \mathsf{LID}(h)$.

Diffusion

Full block: Upon $(\textsc{DiffFB}, h, b)$ from honest P:

1. Require: $\mathsf{match}(h, b)$ and $\neg\mathsf{hasHdr}(P, \mathsf{LID}(h))$.
2. Bookkeeping: $\mathsf{HdrsAdd}(h, \mathbb{T}, P)$, $\mathsf{prefHdr}_P[\mathsf{LID}(h)] := h$, and, if b is non-empty, $\mathsf{BdysAdd}(h, b, \mathbb{T}, P)$.

Header: Upon $(\textsc{DiffHdr}, h)$ from honest P:

1. Require: $\neg\mathsf{hasPoE}(P, \mathsf{LID}(h))$.
2. Bookkeeping: $\mathsf{HdrsAdd}(h, \mathbb{T}, P)$, and if $\mathsf{prefHdr}_P(\mathsf{LID}(h)) = \perp$, set it to h.

Fetching

Headers: Upon $(\textsc{FetchHdrs})$ from honest P:

1. Initialize "to-be-delivered" set $D \leftarrow \emptyset$.
2. For all $(h, t, \mathcal{P}) \in \mathsf{Hdrs}$ where $P \notin \mathcal{P}$:
   (a) If $\mathbb{T} \geq t + \Delta_{\mathsf{hdr}}$ and $\neg\mathsf{hasPoE}(P, \mathsf{LID}(h))$, add h to D.
3. Output $(\textsc{FetchHdrs}, P)$ to $\mathcal{A}$ and get a set $D_{\mathcal{A}}$ from $\mathcal{A}$.
4. Remove from $D_{\mathcal{A}}$ all h with $\mathsf{hasPoE}(P, \mathsf{LID}(h))$.
5. Output $(\textsc{FetchHdrs}, D \cup D_{\mathcal{A}})$ to P.

Bodies: Upon $(\textsc{FetchBdys})$ from honest P:

1. Initialize "to-be-delivered" set $D \leftarrow \emptyset$.
2. For all $(h, b, t, \mathcal{P}) \in \mathsf{Bdys}$ with $P \notin \mathcal{P}$:
   (a) Add (h, b) to D if the following conditions are met:
      i. $\mathsf{prefHdr}_P[\mathsf{LID}(h)] = h$;
      ii. for all honest P': $\mathsf{prefHdr}_{P'}[\mathsf{LID}(h)] \in \{h, \perp\}$;
      iii. there exists $k \geq 0$ s.t. (a) $\mathbb{T} \geq t + \beta + k \cdot B/C$ and (b) $\mathsf{higherBdys}(h) \leq k$.
3. Output $(\textsc{FetchBdys}, P)$ to $\mathcal{A}$ and get a set $D_{\mathcal{A}}$ from $\mathcal{A}$.
4. For each $(h, b) \in D_{\mathcal{A}}$, if (a) $\mathsf{match}(h, b)$, (b) $\neg\mathsf{hasBdy}(P, \mathsf{LID}(h))$, and (c) $\mathsf{prefHdr}_P(\mathsf{LID}(h)) = h$: add (h, b) to D.
5. For each $(h, b) \in D$, $\mathsf{BdysAdd}(h, b, \mathbb{T}, P)$.
6. Output $(\textsc{FetchBdys}, D)$ to P.

Fig. 3. Network functionality $\mathcal{F}_{\mathsf{diff}}$ capturing the QUEQ model.

The detour via the environment (in Steps (2)–(3)) "externalizes" message validity, which commonly depends on the state of the application layer. The idea is that an honest party verifies a received header and re-inputs it into the network layer if it is valid, thereby contributing to its diffusion.

In order to protect against equivocations, Step (3) is only executed if the party has not already seen multiple versions of the message. Note, however, that the first equivocation is still forwarded to the neighbors, with the goal of spreading a *proof of equivocation (PoE)* (i.e., two different messages for the same lottery ticket) throughout the network.

Body diffusion works as follows:

1. The sender of a message—in addition to sending the header to its neighbors—also announces when it is in possession of the corresponding body.
2. When a party learns of a body announcement corresponding to the *preferred* header, which is the *first* header it has seen for a particular message ID, it downloads that body from the sender of the announcement and in turn announces possession of the body to its neighbors. The party then outputs the body to the environment. Importantly, the parties download bodies belonging to *higher-priority* headers first.

The network functionality. Based on the motivation above, $\mathcal{F}_{\mathsf{diff}}$ (cf. Figure 3) allows for the transmission of messages consisting of a header and a body. Due to their small size, headers are delivered to all parties within a fixed delay

$$\Delta_{\mathsf{hdr}} \text{ after sending}$$

(or after the first honest party has seen them). Also, $\mathcal{F}_{\mathsf{diff}}$ guarantees that if one honest party sees a PoE, all other honest parties see it at most Δ_{hdr} later.

The delivery time of the (much larger) bodies, which for simplicity are all assumed to have the same size B, depends on the capacity and delays of the parties' connections, and on the traffic in the network. Specifically, consider a body b belonging to a header for which there are no equivocations. If there are no higher-priority messages impeding the diffusion of the body, it will be delivered $\beta := d \cdot \left(\frac{B}{C} + D\right)$ time after sending, where d is the diameter of the underlying network graph, C is the capacity of the connection between two peers, and D is an upper bound on the latency between two peers (imposed by the speed of light). The parameter C will be referred to as the "capacity of the network" in this work and represents the optimal throughput achievable in our model.[14] For simplicity and in order for lower-level details such as the diameter d not to pertrude the model abstraction boundary, this expression is captured by a *base delay* β in $\mathcal{F}_{\mathsf{diff}}$.

Taking traffic into account, note that each body b' belonging to a higher-priority header will "cut in front" of b at most once, after which b continues its travel behind b'. The time b spends yielding to b' is at most the time it takes for b' to enter the connection, which is

$$\tau := B/C \ ,$$

where τ is referred to as the *queueing delay*.

Based on the above, b will be delivered to all parties

$$\beta + k \cdot \tau \text{ time after sending}$$

(or after the first honest party has seen it) if at that point at most k higher-priority messages have been submitted to the network. Of course, the adversary may cause a message to be delivered at any time before.

Note that for equivocated messages, the body delivery guarantees are limited: since in the motivating gossip protocol above, parties only download bodies for their preferred headers, global body delivery is only guaranteed if all honest parties have the same preferred header. It is up to the application using $\mathcal{F}_{\mathsf{diff}}$ to ensure that such messages are "not essential"—possibly by using PoEs.

4 Simple Leios

The Leios protocol is a black-box construction of a high-throughput ledger protocol on top of a low-throughput protocol, the *base protocol* Π_{base}. The base protocol is assumed to satisfy persistence (cf. Definition 2), liveness (cf. Definition 3), and ledger quality (cf. Definition 4). Morevoer, it is assumed that the size of payload vectors supported by Π_{base} is large enough to contain the messages the overlay protocol must put on the base ledger.

The bandwidths for the base protocol and the Leios extension are separated: Specifically, the base protocol may require arbitrary hybrid functionalities $\mathcal{F}_i$ to run, including one that models the networking assumptions used by the base protocol. Conversely, functionality $\mathcal{F}_{\mathsf{diff}}$ introduced in Section 3 is used to model the networking assumptions made by the Leios extension, which is designed to be near-optimal in the bandwidth C allocated to $\mathcal{F}_{\mathsf{diff}}$. Nevertheless, since the bandwidth needed for the base protocol is fixed, one can still achieve $1 - \delta$ throughput relative to large enough overall capacity for any $\delta > 0$ (for large enough C).

[14] It is important to note, however, that in the type of gossip networks that motivate our design of $\mathcal{F}_{\mathsf{diff}}$, the network capacity C is *not* equivalent to the upload and download speed C' of individual parties. This discrepancy arises because, in a straightforward implementation of gossip, parties divide their bandwidth equally among their peers. As a result, $C = C'/\delta$, where δ represents the maximum degree in the network. It is conceivable that a more sophisticated gossip design could enable protocols that achieve throughputs up to C'.

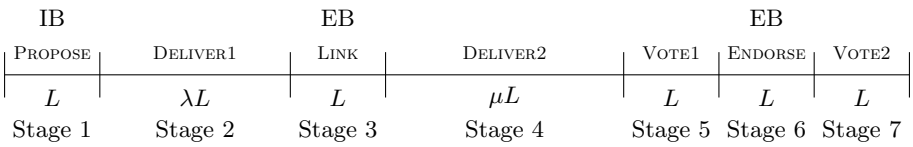

Fig. 4. The seven stages of the Leios pipeline.

This section presents *Simple Leios*, a version of Leios that (1) achieves near-optimal throughput that is scaled by the ledger quality η of the base protocol and (2) has a somewhat undesirable trade-off between liveness delay and error probability. In rough terms, for parameters L and λ, Simplified Leios achieves[15]

- a relative (w.r.t. C) throughput of roughly $\frac{\lambda}{\lambda+1}\alpha_{\mathrm{H}} \min(L/\eta, 1)$,
- settlement and liveness guarantees equal to the maximum of those of the base protocol and $O(\lambda^3 L^3/\beta^2)$,
- security error $\mathcal{O}(t)e^{-\Omega(L)}$, where t is an upper bound on the amount of time the protocol is run, and
- fairness.

Note that λ and L are free parameters serving the following purposes. Scaling parameter λ defines how close the IB data rate is in relation to the network capacity C (setting the IB data rate to approximately $\lambda/(\lambda+1)C$). L controls the probability of bad pipeline events, and by nature of the analysis of Simple Leios, it should be chosen to be at least $\log^2(\kappa)$ in the security parameter κ to give meaningful guarantees.

Section 5 shows an augmented version, *Full Leios*, which achieves near-optimal throughput independent of η and with security error independent of L, meaning liveness delay can be constant (by choosing constant λ and L) while still yielding negligible error. For simplicity, both versions of Leios are first presented assuming that the adversary does not attempt to equivocate any protocol messages. Section 6 shows how to remove this assumption.

4.1 Protocol Overview

Input blocks. Leios is based on the concept of *input endorsing* [24,43,32], in which parties (run local, VRF-based lotteries to obtain the rights to) produce payload-carrying so-called *input blocks (IBs)*, which are then ordered using an underlying (low-throughput) consensus protocol. While the input-endorser concept is natural, implementing this technique, is non-trivial and requires considerable care, particularly in a realistic network model

One issue is that IBs can only be included in the underlying low-throughput ledger *by reference*. However, this leads to a data availability issue since the adversary may include references for which there are no corresponding IBs. Such behavior can be foiled by certifying IB availability via voting and modifying the ledger rules of the base protocol to only accept certified IBs.

Endorser blocks. Unfortunately, due to the high rate at which IBs need to be produced in order to achieve large throughput, certifying each IB individually would require too much computational overhead as well as space in the ledger of the (low-throughput) base protocol. To that end, IBs are first collected (by reference) in so-called *endorser blocks (EBs)* (also produced based on local, VRF-based lotteries), which are then certified. The idea is that parties contribute to the certification of an EB if they have seen all IBs referenced therein.

Fairness. A blockchain protocol is *fair* if the fraction of honestly created blocks in the chain corresponds to the actual fraction of honest stake (or whatever other resource underlies the security of the protocol). This extends to the input endorser technique in the sense that the fraction of honest IBs ultimately included via the base protocol should correspond to the fraction of honest stake.

The adversary may negatively affect the fairness of an input-endorser protocol by only referencing adversarially generated IBs in adversarial EBs. For example, if EBs are produced at a rate equal to the capacity of

[15] Recall that α_{H} denotes the fraction of honest stake.

the base ledger, then such an adversarial strategy leads to roughly 75% adversarial IBs in settings where the adversarial stake is only slightly below 50%. It is therefore necessary to produce EBs at a rate high enough such that there is at least one honest EB for every block in the base protocol.

However, at such a high rate, not all EBs can be included in the low-throughput base protocol, and including only a subset of all EBs will not solve the fairness problem since it is unclear which EBs are from honest parties. Leios tackles this issue by suitably referencing older EBs from within newer EBs.

Freshest-first delivery. As explained in Section 1, IBs are diffused in a *freshest-first* fashion in order to prevent the attacker from spamming the network by releasing old IBs in bulk, i.e., to defend against message bursts. This is modeled by using time as the priority notion in $\mathcal{F}_{\mathsf{diff}}$ (cf. Section 3). This approach comes with its own challenges, however, as special care has to be taken to ensure that IBs arrive before certain deadlines.

The pipeline. The Leios design is a pipelined one. A pipeline instance of Simple Leios consists of seven stages (cf. Figure 4). Pipeline instances run concurrently, with a new instance beginning every L slots, where L referred to as the *slice length*. To accommodate our equivocation-handling mechanism (cf. Section 6), we require that $L \geq 5\Delta_{\mathsf{hdr}}$ (where Δ_{hdr} is the header delay in $\mathcal{F}_{\mathsf{diff}}$). IBs and EBs are generated at a steady rate throughout the execution, where each IB plays a "proposer" role in the most recent pipeline instance, and each EB plays a "linking" role and an "endorsing" role in two different pipeline instances. The pipeline stages, whose lengths (in slots) are multiples $\{1, \lambda, \mu\}$ of L (where μ is another parameter defined later), are as follows:

1. Propose (length L): IBs produced in this stage are the "focus" of the pipeline instance;
2. Deliver1 (length λL): time for IBs from the Propose stage to be diffused;
3. Link (length L): IBs from the Propose stage are referenced by EBs produced in this stage;
4. Deliver2 (length μL): time for adversarial IBs referenced by the EBs from the Link stage to diffuse;
5. Vote1 (length L): parties cast votes for EBs from the Link stage whose referenced IBs have been delivered; if an EB obtains a number of Vote1-votes above a certain threshold, it becomes Vote1-certified;
6. Endorse (length L): EBs from the Link stage that have garnered a Vote1-certificate are referenced by EBs from this stage;
7. Vote2 (length L): parties cast votes for EBs from the Endorse stage that satisfy the following property: the EB references (i) Vote1-certified EBs only and (ii) a majority of all Vote1-certified Link-stage EBs; if an EB obtains a number of Vote2-votes above a certain threshold, it becomes Vote2-certified.

Pipeline guarantees. The goal of every pipeline is to produce at least one Vote2-certified EB while ensuring that each Vote2-certified EB, via references to Vote1-certified EBs, covers all honest IBs from the Propose stage. Including any such EB per pipeline into the underlying low-throughput protocol will thus ensure liveness of honest input blocks as well as fairness.

Delivering IBs in time for EB inclusion. The (honest) IBs generated in the Propose stage must be delivered to EB producers before the Link stage begins. To that end, Propose is followed by a Deliver1 stage, in which these IBs propagate through the network. However, the Deliver1 stage overlaps with the Propose stages of subsequent pipeline instances, and the IBs produced there take precedence because $\mathcal{F}_{\mathsf{diff}}$ delivers IBs in a *freshest-first* manner. To guarantee that this interference does not delay the original IBs by too much, the overall IB rate must be such that the Deliver1 stage, which is of length λL, can handle the IB traffic originating during both the Propose and Deliver1 stages. This is ensured by bounding the IB data rate (relative to the capacity of the network) by $\lambda/(\lambda + 1)$.

Note that thus, there is a tradeoff between higher throughput (larger λ) and shorter pipeline (smaller λ). The error $e^{-\Omega(L)}$ stems from a Chernoff bound guaranteeing that the number of IBs is close to its expectation.

Delivering IBs in time for first EB voting. Observe that honest EBs produced during Link may only garner enough votes in Vote1 if all the IBs they reference are globally delivered before Vote1 starts. While honest IBs are guaranteed to be globally delivered already before Link (as argued above), the adversary may start diffusing all of its IBs from Propose late, in the worst case just before the end of Deliver1. As a consequence, some honest EB producers may include those adversarial IBs, and therefore a second delivery period, Deliver2, is required for them to be diffused globally before the start of Vote1 as otherwise, these honest EBs may be at risk of being lost due to not receiving enough votes.

In a vein similar to Deliver1 above, the delivery of said adversarial IBs is ensured if the (relative) IB data rate is below $(\mu + 1)/(\lambda + \mu + 2)$, which essentially guarantees that all the IB traffic of the first four stages fits into the Link and Deliver2 stages. In order not to further restrict the throughput, one requires $\lambda/(\lambda + 1) \leq (\mu + 1)/(\lambda + \mu + 2)$, which is satisfied for $\mu = \Theta(\lambda^2)$.

All honest EBs certified in time. The above guarantees that all honest EBs produced in Link will be (globally seen as) Vote1-certified before the beginning of the Endorse stage (assuming $L \geq \Delta_{\mathsf{hdr}}$—implied by the above assumption that $L \geq 5\Delta_{\mathsf{hdr}}$). They will therefore be referenced by all honest EBs produced in Endorse.

All honest Endorse EBs will be Vote2-certified since (i) they only reference Vote1-certified Link EBs and (ii) they all reference a majority of Vote1-certified Link EBs since all honest Link EBs are certified as argued above (and honest parties control the majority of stake).

Therefore, by the end of the pipeline, there will be at least one Vote2-certified EB that, via references to Vote1-certified EBs, covers all honest IBs from the Propose stage.

Actual availability of the payload. One final time, the adversary may introduce a delay, this time on the availability of the payload, by adopting the following (worst-case) strategy: just before the end of Deliver2, it serves some IBs (that it was supposed to diffuse during Propose), along with an EB that references them, to a small subset of honest parties, just enough to obtain sufficiently many Vote1 votes to get the EB Vote1-certified. The EB will subsequently be picked up by honest EBs in the Endorse stage, and therefore the adversarial IBs end up referenced in the base ledger. However, they still need to be diffused to most honest parties, while also competing with newer IBs.

In the above spirit, these IBs will be delivered after an imaginary Stage $5'$ of length νL if the (relative) IB data rate is below $\nu/(2 + \lambda + \mu + \nu)$. Again, in order not to further restrict the throughput, one requires $\lambda(\lambda + 1) \leq \nu/(2 + \lambda + \mu + \nu)$, which is satisfied for $\nu = \Theta(\lambda^2 + \lambda\mu) = \Theta(\lambda^3)$.

Ledger construction. To extract the ledger, a party can parse the ledger of the base chain, and list all referenced (Vote2-certified) EBs by order of appearance; from this list, the party can extract a list of Link-stage EBs (also in order of appearance), and sequence all IBs referenced therein, finally defining an ordered list of ledger transactions.

While deriving the ledger in this fashion, some transactions may become invalid due to conflicts with preceding transactions and must therefore be dropped; for simplicity, this aspect is ignored here and it is assumed that the environment provides transactions in a way that avoids conflicts. Moreover, since the transaction order is not fully known before settlement, all computation required for validating a transaction must generally be performed once it is referenced via an EB included in the base ledger. This requirement is especially relevant for account-based systems, such as Ethereum's EVM, even though concurrent smart-contract execution [18] may help mitigate some of the associated cost.

Interestingly, this validation bottleneck can be avoided in UTxO-based ledgers (e.g., Bitcoin or Cardano, the latter using the EUTxO model [12]). In such ledgers, verifying a transaction's correctness typically involves two steps: (1) checking that all inputs exist and are unspent, and (2) verifying that, given those inputs, the script execution produces the correct outcome. While the first step (which is computationally cheap) can only be performed after serialization is complete, the second step (which is more expensive) can be executed "optimistically" before serialization. In our context, this occurs during IB validation, meaning the bulk of the validation workload is distributed over time during IB production, thus avoiding computational spikes on the client side.

4.2 Protocol Messages

It will be useful to define a function that returns the slice x slices before the slice that contains some slot s. Specifically, the function returns the interval

$$\mathsf{slice}_L(s, x) \;:=\; [s', s' + (L-1)]\ ,$$

for $s' :=\ \lfloor s - xL \rfloor_L$, where $\lfloor \cdot \rfloor_L$ rounds down to the nearest multiple of L.

Input blocks. An input block $\mathsf{IB} = (h, b)$ consists of a header h and a body b. The header $h = (\text{“IB”}, s, P, \pi, x, \sigma)$ consists of a slot number s, a stake holder ID P, a lottery proof π (created using $\mathcal{F}_{\mathsf{vrf}}$), a body hash x, and a signature σ on h (without σ). The body contains a list L_T of ledger transactions.

For the purpose of sending them via $\mathcal{F}_{\mathsf{diff}}$, set $\mathsf{LID}(h) := (\text{“IB”}, s, P)$—which also makes the lottery identifiers totally ordered—and $\mathsf{match}((\cdot,\cdot,\cdot,\cdot,x,\cdot), b)$ if and only if x is the hash of b.

IBs are created at rate f_{I}, for some parameter f_{I}: given relative stake α, for a given slot, a stakeholder P is assigned the right to produce $\phi_{\mathrm{I}}(\alpha) = \alpha f_{\mathrm{I}}$ input blocks in expectation. Note that, for values of $f_{\mathrm{I}} > 1$, one needs to subdivide each slot into $m > f_{\mathrm{I}}$ subslots, each with $\phi_{\mathrm{I}}(\alpha) = \alpha f_{\mathrm{I}}/m$. For simplicity, and WLOG, we assume $f_{\mathrm{I}} \leq 1$ in the remainder of this work.

An input block is *valid* if lottery proof π is valid for (s, P) and signature σ is valid for P.[16]

Endorser blocks. Endorser blocks $\mathsf{EB} = (\text{“EB”}, s, P, \pi, L_{\mathsf{I}}, L_{\mathsf{E}}, \sigma)$ consist of a slot number s, a stake holder ID P, a lottery proof π (created using $\mathcal{F}_{\mathsf{vrf}}$), lists L_{I} and L_{E} of IB and EB references, respectively, and a signature σ on EB (without σ). Due to their small size, especially compared to IBs, EBs are modeled as “header-only” for the purpose of sending them via $\mathcal{F}_{\mathsf{diff}}$, where $\mathsf{LID}(h) := (\text{“EB”}, s, P)$.

EBs are created at rate f_{E}, for some parameter f_{E}: given relative stake α, for a given slot, a stakeholder P is assigned the right to produce $\phi_{\mathrm{E}}(\alpha) = \alpha f_{\mathrm{E}}$ endorser blocks in expectation.[17]

An EB is *valid* if (a) lottery proof π is valid for (s, P) and signature σ is valid for P, (b) L_{I} only references IBs from $\mathsf{slice}_L(s, \lambda+1)$, and (c) L_{E} only references EBs from $\mathsf{slice}_L(s, \mu+2)$. Note that EB validity can only be determined once all referenced IBs and EBs have been received.

Vote messages. Parties use $\mathcal{F}_{\mathsf{sbv}}$ to generate voting strings for messages in different “periods” j. The periods used in Leios have the format $j = (s, x)$, where $x \in \{1, 2\}$, indicating whether the vote is for Vote1 or Vote2-certification.[18] Intuitively, the $\mathcal{F}_{\mathsf{sbv}}$ guarantees that (i) if all honest parties are instructed to vote for a particular message m in a period j, enough votes are generated to create a valid period-j certificate, and (ii) if no honest party votes for m in period j, the adversary cannot create a period-j certificate for m. Note that $\mathcal{F}_{\mathsf{sbv}}$ can output $\perp$ to a party P in a period j, which means that P is not to cast a vote message in period j.

A vote message has the format $(\text{“vote”}, j, P, h, v)$, where j is a period number, P is stake holder ID, h is an EB reference, and $v \neq \perp$ is vote string output by $\mathcal{F}_{\mathsf{sbv}}$ for message h and period j.[19] Similarly to EBs, votes are also modeled as “header-only” for the purpose of sending them via $\mathcal{F}_{\mathsf{diff}}$, with $(\text{“vote”}, j, P)$ as the lottery ID.

It is important to note that, in contrast to IBs and EBs, which are produced over the course of the respective stage, all votes of a given stage (Vote1 and Vote2) are initiated right at the beginning of the stage.[20]

Validity. In the following sections, it is tacitly assumed that parties drop all protocol messages that are not valid as defined in this section.

Protocol $\pi_{\mathsf{simpleLeios}}$

Parameters: L: slice length; λ, μ: stage-length multipliers for Deliver1 and Deliver2 stages, respectively; $f_{\mathrm{I}}, f_{\mathrm{E}}$: frequencies of IBs and EBs, respectively (implicit in this figure).

General

Initialization: Upon receiving (Stake, SD) from the base protocol, store SD and output (Stake, SD) to the environment. Upon receiving (Init, V_{Ext}) from the environment, request keys from $\mathcal{F}_{\mathsf{sig}}$, $\mathcal{F}_{\mathsf{vrf}}$, and $\mathcal{F}_{\mathsf{sbv}}$. Exchange these keys with other parties via the key registration functionality to obtain a list of keys MI. Use this list to generate predicate V_{chkCerts} (which has access to those functionalities for verification purposes).[a] Send input (Init, V_{chkCerts}) to the base protocol.

Network and ledger: At the beginning of each slot:

1. Fetch new messages (IBs, EBs, and votes) from $\mathcal{F}_{\mathsf{diff}}$ and discard invalid ones (cf. Section 4.2), using $\mathcal{F}_{\mathsf{sig}}$, $\mathcal{F}_{\mathsf{vrf}}$, and $\mathcal{F}_{\mathsf{sbv}}$ for verification. Re-input valid messages into $\mathcal{F}_{\mathsf{diff}}$ (cf. Section 3).
2. Compute the ledger by following the EB-(EB-)*-IB references in the base ledger in order and concatenating the payloads in the IBs up to the point of the first IB with an non-downloaded body (if any).
3. Output the resulting ledger upon receiving command FtchLdg.

Base chain: In each slot s (upon receiving (FtchPld) from the base protocol):

1. Pick an arbitrary Vote2-certified EB EB from $\mathsf{slice}_L(s, 2)$ and reply (FtchPld, EB) to the base protocol
2. If there is no such EB, issue (FtchPld) to the environment, receive (FtchPld, $\boldsymbol{p}$), and reply (FtchPld, $\boldsymbol{p}$) to the base protocol.

Protocol Roles

In every slot s, each party P executes the following roles:

IB Role:

1. Check, using $\mathcal{F}_{\mathsf{vrf}}$, if eligible to produce an IB in slot s. If so, let π be the corresponding proof; otherwise, exit the procedure.
2. Issue (FtchPld) to the environment and receive (FtchPld, $\boldsymbol{p}$).
3. Set $\mathsf{IB} := (h, \boldsymbol{p})$ where $h :=$ ("IB", s, P, π, x, σ), where x is the hash of b and σ is a signature, obtained from $\mathcal{F}_{\mathsf{sig}}$, of (the rest of) h under P's signing key in MI.
4. Diffuse IB via $\mathcal{F}_{\mathsf{diff}}$.

EB Role:

1. Check if eligible to produce an EB in slot s. If so, let π be the corresponding proof; otherwise, exit the procedure.
2. Link role: set L_{I} to be all (completely downloaded) IBs from $\mathsf{slice}_L(s, \lambda + 1)$.
3. Endorse role: set L_{E} to be all Vote1-certified EBs from $\mathsf{slice}_L(s, \mu + 2)$.
4. Diffuse $\mathsf{EB} :=$ ("EB", $s, P, \pi, L_{\mathsf{I}}, L_{\mathsf{E}}, \sigma$) via $\mathcal{F}_{\mathsf{diff}}$, where σ is a signature, obtained from $\mathcal{F}_{\mathsf{sig}}$, of (the rest of) EB under P's signing key in MI.

Vote-1 Role: If s is the first slot of $\mathsf{slice}_L(s, 0)$:

1. Check, using $\mathcal{F}_{\mathsf{sbv}}$, if eligible to produce Vote1-vote in slot s.
2. Create a vote for each EB from $\mathsf{slice}_L(s, \mu + 1)$ for which all referenced IBs are fully downloaded.
3. Diffuse the votes via $\mathcal{F}_{\mathsf{diff}}$.

Vote-2 Role: If s is the first slot of $\mathsf{slice}_L(s, 0)$:

1. Check, using $\mathcal{F}_{\mathsf{sbv}}$, if eligible to produce Vote2-vote in slot s.
2. Create vote for each EB from $\mathsf{slice}_L(s, 1)$ that references (i) a majority of all seen Vote1-certified EBs from $\mathsf{slice}_L(s, \mu + 3)$ and (ii) no other EBs.
3. Diffuse the votes via $\mathcal{F}_{\mathsf{diff}}$.

[a] Note that the extension protocol ignores the validation predicate V_{Ext} received via the Init command. These are "ledger rules" for the high-level protocol and could be used to sanitize the ledger before outputting it. For simplicity, this is ignored here.

Fig. 5. The Simple Leios Protocol.

4.3 The Protocol

The Simple Leios protocol is summarized in Figure 5. Initially, parties obtain a stake distribution from the base protocol and locally generate keys for the IB/EB lottery as well as for voting, which they then register to the key registration functionality (cf. Section 2.1), that in turn distributes the public key list MI to all parties. Then, each party takes the voting public keys registered and uses them to compute a verification function $V_{\mathsf{chkCerts}}(\boldsymbol{p})$ that accepts if and only if (i) $\boldsymbol{p}$ consists of pairs (EB, c), where c is a valid certificate for EB under the voting keys, and (ii) these EBs are from strictly increasing pipelines. This predicate is then passed to the base protocol.

Subsequently, in each slot, every party checks whether they are eligible to engage in any of the following roles:

– *IB role:* Creating IBs, corresponding to the Propose stage.

[16] Note that x being the hash of b is already guaranteed by $\mathcal{F}_{\mathsf{diff}}$ with the above match predicate.
[17] Similarly to IBs, rates $f_{\mathrm{E}} > 1$ require subslots.
[18] Thus, the periods are totally ordered.
[19] Note that v is tied to j, P, and h by $\mathcal{F}_{\mathsf{sbv}}$; hence, no additional signature on the vote message is needed.
[20] Possibly delaying some stage-Vote2 votes by Δ_{hdr} in case a respective EB is produced late in the Endorse stage.

– *EB role:* Creating EBs; this role can be subdivided into a link role and an endorse role, corresponding to the LINK and ENDORSE stages, respectively.
– *Vote-1 role:* Voting for EBs according to the VOTE1 stage.
– *Vote-2 role:* Voting for EBs according to the VOTE2 stage.

To illustrate this, assume $\lambda = \mu = 1$. In that case, there are seven active pipelines at any point, each of which is in a different stage. Assume they are numbered from 1 to 7 in ascending order of age (i.e., the newest one is pipeline 1). Given this, a party plays the following roles in each pipeline: *Pipeline 1:* IB role; *Pipeline 2:* no role; *Pipeline 3:* link role; *Pipeline 4:* no role; *Pipeline 5:* vote-1 role; *Pipeline 6:* endorse role; *Pipeline 7:* vote-2 role.

Parties continuously input a most recent VOTE2-certified endorser block to the base protocol, resulting in roughly every (η/L)-th pipeline being represented in the base chain. If there is no certified EB available (due to low participation—should the base protocol allow for dynamic participation), the parties input, as payloads to the base protocol, transactions received from the environment.[21]

4.4 Analysis

Analysis overview. It is easy to see that under low participation, Leios inherits safety and liveness from the base protocol (because whenever there are no certified EBs available, parties input transactions directly to the base protocol); the remainder of this section therefore assumes full (honest) participation.[22]

The main part of the analysis is to show that with high probability, a *pipeline instance* is *successful*, which intuitively means that (1) at least roughly $\alpha_{\mathrm{H}} L f_{\mathrm{I}}$ honest IBs are produced (during the PROPOSE stage),[23] (2) at least one VOTE2-certified EB is produced, and (3) any VOTE2-certified EB references all honest IBs. Intuitively, (3) guarantees that roughly every (η/L)-th pipeline is represented in the base chain—as parties continuously input a most recent VOTE2-certified EB to the base protocol; (2) ensures that such an EB exists, and (1) ensures that enough honest IBs are available per pipeline to achieve the desired throughput.

The success of entire pipeline instances can be related to the success of each of the $5 + \lambda + \mu$ constituting slices; a *slice* is successful if (a) *at least* roughly $\alpha_{\mathrm{H}} L f_{\mathrm{I}}$ honest IBs are produced, (b) *at most* roughly $L f_{\mathrm{I}}$ IBs are produced overall, and (c) an honest majority of EBs are produced during the slice. Criterion (a) relates directly to (1); (b) ensures that the network is not too congested and the delivery guarantees of $\mathcal{F}_{\mathsf{diff}}$ can be used to suitably bound IB delivery times, which is necessary for (2); and (c) helps guarantee (3). The probability $\varepsilon_{\mathsf{fail}}(\varepsilon, f_{\mathrm{I}}, f_{\mathrm{E}}, L) = e^{-\varepsilon^2 \Omega(f_{\mathrm{I}} L)} + e^{-\varepsilon^2 \Omega(f_{\mathrm{E}} L)}$ that a slice is not successful is upper bounded using a standard Chernoff argument, where ε with $0 < \varepsilon < 2\alpha_{\mathrm{H}} - 1$ is the concentration error in the Chernoff bound.

In order to show (2), the proof follows the intuition outlined in Section 4.1, which yields the following parameter choices:

$$f_{\mathrm{I}} \;\approx\; \frac{\lambda}{\lambda + 1} \cdot \frac{C}{B} \quad \text{and} \quad \mu \;\approx\; \lambda^2 \,.$$

Note that the above choice of f_{I} already suggests that the IB data rate $f_{\mathrm{I}} B$ approaches C for large λ. Furthermore, also as explained in Section 4.1, the final liveness delay is dominated by $\nu := \mathcal{O}(\lambda^3)$.

Successful pipelines. A successful pipeline produces at least one VOTE2-certified EB that covers all PROPOSE-stage IBs:

Definition 6. *Let $\varepsilon > 0$. A* pipeline instance *is* ε-successful *if*

1. *its* PROPOSE *stage produces at least $\alpha_{\mathrm{H}}(1-\varepsilon) L f_{\mathrm{I}}$ honest IBs,and*
2. *its* ENDORSE *stage produces an honest EB that gets* VOTE2*-certified by the end of the* VOTE2 *stage, and*
3. *by the end of the* VOTE2 *stage, all honest input blocks from the* PROPOSE *stage are (indirectly) referenced* by all VOTE2*-certified EB from the* ENDORSE *stage.*

[21] We further expand on this idea in Appendix F.3.
[22] Note that this means that all honest parties are assumed to be online, but *not* that all parties are honest.
[23] Recall that α_{H} denotes the fraction of honest stake.

Recall that the protocol execution is divided into slices of length L. In order for a slice to be "useful," the number of IBs generated during it must be in a reasonable interval, and an honest majority of the EBs generated must be from honest parties. This is captured by the following definition:

Definition 7. *Let $\varepsilon > 0$. A* slice *is* ε-successful *if it produces*

(1) at least $\alpha_{\mathrm{H}}(1-\varepsilon)Lf_{\mathrm{I}}$ honest IBs,

(2) no more than $(1+\varepsilon)Lf_{\mathrm{I}}$ IBs, and

(3) an honest majority of endorser blocks.

The next step is to upper bound the probability of an unsuccessful slice as stated in Lemma 1.

Lemma 1. *Let ε such that $0 < \varepsilon < 2\alpha_{\mathrm{H}} - 1$. A slice is ε-successful except with probability*

$$\varepsilon_{\mathsf{fail}}(\varepsilon, f_{\mathrm{I}}, f_{\mathrm{E}}, L) := e^{-\varepsilon^2\Omega(f_{\mathrm{I}}L)} + e^{-\varepsilon^2\Omega(f_{\mathrm{E}}L)} \ .$$

Proof. The upper bound follows from a union bound over the three properties of a successful slice and the Chernoff-Hoeffding bound as given in Appendix B as follows.

1. Item 1. Let $X_{ij} \in \{0,1\}$ be the indicator random variable that honest party j gets the right to produce an IB in slot i (aka slot leadership), let there be n_H honest parties, where the honest party with index $j \in [1, n_H]$ holds relative stake α_j, and let

   $$X \ = \ \sum_{i=1}^{L}\sum_{j=1}^{n_H} X_{ij}$$

   be the random variable representing the number of honest slot leaderships during the given slice. Since $\Pr[X_{ij}] = \alpha_j f_{\mathrm{I}}$,

   $$E[X] = \alpha_{\mathrm{H}} f_{\mathrm{I}} L \ ,$$

   and, by the Chernoff-Hoeffding bound from Appendix B, the probability that Property (1) is violated is thus

   $$P_1 \ \leq \ \exp\left(-\frac{\varepsilon^2\alpha_{\mathrm{H}} f_{\mathrm{I}} L}{2}\right).$$

2. In expectation, $f_{\mathrm{I}}L$ IBs are generated during the slice. By the Chernoff-Hoeffding bound, along the lines of the above, the probability that Property (2) is violated is thus

   $$P_2 \ \leq \ \exp\left(-\frac{\varepsilon^2 f_{\mathrm{I}} L}{3}\right).$$

3. In expectation, a slice produces $\alpha_{\mathrm{H}} f_{\mathrm{E}} L$ honest endorser blocks, and at most $(1-\alpha_{\mathrm{H}}) f_{\mathrm{E}} L$ adversarial ones. The probability that there are at least as many adversarial as honest endorser blocks can thus be estimated by a union bound over the probabilities that the honest production undercuts its expectation by a factor of $(1-\varepsilon)$ and the adversarial production exceeds its expectancy by a factor of $(1+\varepsilon)$, as long as $(1-\varepsilon)\alpha_{\mathrm{H}} > (1+\varepsilon)(1-\alpha_{\mathrm{H}})$, which is equivalent to $\varepsilon < 2\alpha_{\mathrm{H}} - 1$. Thus,

   $$P_3 \ \leq \ \exp\left(-\frac{\varepsilon^2\alpha_{\mathrm{H}} f_{\mathrm{E}} L}{2}\right) + \exp\left(-\frac{\varepsilon^2(1-\alpha_{\mathrm{H}}) f_{\mathrm{E}} L}{3}\right) \ .$$

□

Before proceeding, recall that $\mathcal{F}_{\mathsf{diff}}$ (see Section 3) guarantees the delivery of an IB within $\beta + k \cdot \tau$ time. Here, β represents the base delay for transmitting a message through the network (assuming no queuing), and $k \cdot \tau$ accounts for the maximum queuing delay, where τ is the time required to download the IB over a single communication link, and k is the number of fresher IBs in transit.

In order to analyze the success of an entire pipeline instance, consider the following parameter settings (cf. Section 4.1 for intuition):

$$f_{\mathrm{I}} \;:=\; \frac{\lambda-\beta'}{(1+\varepsilon)(\lambda+1)}\cdot\frac{1}{\tau}\;, \quad \text{and} \quad \mu \;:=\; \left\lceil\frac{\lambda^2+\lambda-\beta'}{1+\beta'}\right\rceil\;, \tag{1}$$

where β' is such that $\beta' L = \beta$.

The following lemma "reduces" the notion of pipeline success to the successfulness of slices.

Lemma 2. *For any $\varepsilon > 0$, a pipeline instance is successful if all its slices are ε-successful.*

Proof. It is sufficient to demonstrate the following facts:

(1) all honest IBs are delivered by the end of the Deliver1 stage; and

(2) all IBs seen by an honest party by the end of the Deliver1 stage are seen by all honest parties by the end of the Deliver2 stage.

Success of the Link stage then guarantees that all honest IBs from the Propose stage are referenced by all honest Vote1-certified EBs from the Link stage, and success of the Endorse stage guarantees that an (honest) Vote2-certified EB is produced that covers a majority of all Vote1-certified EBs of the Link stage. The latter implies that at least one honest Link-stage EB is referenced (as there is a majority of honest parties).

Fact (1) is proved by ensuring that the IB production during the first $\lambda+1$ pipeline slices essentially "fits" into the λL slots of the Deliver1 stage. Specifically, consider any honest IB from the Propose stage. As per the guarantees of $\mathcal{F}_{\mathsf{diff}}$, the IB is globally delivered

$$\beta + k\cdot\tau \text{ slots after sending}$$

if at that point there are at most k IBs with newer slot numbers in the network.[24] Since all slices in the pipeline are assumed to be ε-successful, there are at most $(1+\varepsilon)(\lambda+1)Lf_{\mathrm{I}}$ IBs produced during all of Propose and Deliver1. Therefore, global delivery is guaranteed if

$$\beta' L + \big((1+\varepsilon)(\lambda+1)Lf_{\mathrm{I}}\big)\tau \;\overset{!}{\leq}\; \lambda L\;,$$

using $\beta = \beta' L$. This is satisfied if

$$f_{\mathrm{I}} \;\leq\; \frac{\lambda-\beta'}{(1+\varepsilon)(\lambda+1)}\cdot\frac{1}{\tau}\;,$$

which corresponds to the choice of f_{I} above.

One similarly argues that Fact (2) is true if the IB production during the first $\lambda+\mu+2$ slices fits into the $\mu+1$ slices corresponding to the Link and Deliver2 stages. This yields the condition

$$\beta' L + \big((1+\varepsilon)(\lambda+\mu+2)Lf_{\mathrm{I}}\big)\tau \;\overset{!}{\leq}\; (\mu+1)L\;,$$

which is satisfied if

$$f_{\mathrm{I}} \;\leq\; \frac{(\mu+1)-\beta'}{(1+\varepsilon)(\lambda+\mu+2)}\cdot\frac{1}{\tau}\;.$$

Given the choice of f_{I}, this is satisfied if μ is chosen such that

$$\frac{\lambda-\beta'}{\lambda+1} \;\leq\; \frac{(\mu+1)-\beta'}{\lambda+\mu+2}\;,$$

which requires

$$\mu \;\geq\; \frac{\lambda^2+\lambda-\beta'-1}{\beta'+1}\;.$$

This is satisfied by the choice of μ above. □

[24] Recall that τ is the time it takes to send a body over a single "hop" in the network.

Recall from Section 4.1 that the adversary can delay the availability of adversarial IBs referenced (indirectly) by Vote2-certified EBs. The following lemma quantifies this delay.

Lemma 3. *Let ε such that $0 < \varepsilon < 2\alpha_{\mathrm{H}} - 1$. Any IB (indirectly) referenced in a* Vote2*-certified EB from slot s is downloaded by every honest party no later than in slot $s + \nu L$ for*

$$\nu \;:=\; \left\lceil \frac{(\lambda - \beta')(\lambda + \mu + 2) + (\lambda + 1)\beta'}{\beta' + 1} \right\rceil \;=\; \mathcal{O}(\lambda^3/\beta'^2)\,,$$

except for error probability $\varepsilon_\nu = \exp(-\varepsilon^2 f_{\mathrm{I}} \nu L/3)$.

Proof. Note that Any IB referenced by a Vote2-certified EB is available for download from an honest party by the end of the Deliver2 stage of that pipeline. Along the lines of the argumentation in the proof of Lemma 2, we demand that input block data available at the onset of the Vote1 to any honest party will be downloaded by every honest party within the next ν slices.

Thus, let us consider the time interval spanning the first four stages of a production pipeline (Propose to Deliver2) plus ν slots after the Deliver2 stage—of length $2 + \lambda + \mu + \nu$. Applying a Chernoff bound along the lines of Lemma 1, we observe that, except for error ε_ν, no more than $(1 + \varepsilon) f_{\mathrm{I}}(2 + \lambda + \mu + \nu)L$ IBs are produced during this interval. We demand that the respective production of IBs can be downloaded during the last ν slots of the interval:

$$\beta + \big((1 + \varepsilon) f_{\mathrm{I}}(2 + \lambda + \mu + \nu)L\big)\tau \overset{!}{\leq} \nu L\,. \tag{2}$$

Equation (2) is satisfied if

$$f_{\mathrm{I}} \;\leq\; \frac{\nu - \beta'}{(1 + \varepsilon)(\lambda + \mu + \nu + 2)} \cdot \frac{1}{\tau}\,.$$

Given the choice of f_{I}, this is satisfied if ν is chosen such that

$$\frac{\lambda - \beta'}{\lambda + 1} \;\leq\; \frac{\nu - \beta'}{\lambda + \mu + \nu + 2}\,,$$

which requires

$$\nu \;\geq\; \frac{(\lambda - \beta')(\lambda + \mu + 2) + (\lambda + 1)\beta'}{\beta' + 1}\,,$$

which corresponds to the choice of ν, and the lemma follows. □

Main theorems. In the following, let t be the time the protocol is run, $\varepsilon_{\mathsf{fail}}(\varepsilon, f_{\mathrm{I}}, f_{\mathrm{E}}, L)$ the probability a slice fails (cf. Lemma 1). Further, assume the base protocol satisfies persistence with parameter w and ledger quality η with liveness u; let $\varepsilon_{\mathsf{base}}(t)$ be the error of the base protocol when run for time t. Finally, Let $n_{\mathsf{PL}} := 5 + \lambda + \mu = \mathcal{O}(\lambda^2/\beta')$ denote the number of slices per pipeline, and $t_{\mathsf{PL}} := n_{\mathsf{PL}} L$ the length of a pipeline in slots.

Theorem 1. *Let ε be such that $0 < \varepsilon < 2\alpha_{\mathrm{H}} - 1$, and assume the base protocol Π_{base} (run over time t) achieves, except for error $\varepsilon_{\mathsf{base}}(t)$,*

- *persistence with parameter w and*
- *ledger quality with parameters η and u.*

Then, Simple Leios achieves

- *persistence with parameter $w' = w$, and*
- *liveness with parameters $r' = 2L + t_{\mathsf{PL}} + \eta$ and $u' = (2 + \nu)L + t_{\mathsf{PL}} + u$,*

except for error $\varepsilon_{\mathsf{base}}(t) + \mathcal{O}(t)\varepsilon_{\mathsf{fail}}(\varepsilon, f_{\mathrm{I}}, f_{\mathrm{E}}, L)$.

Proof. Since $\mathcal{F}_{\mathsf{sbv}}$ guarantees that there are no forgeries, the overlay protocol inherits persistence from the base protocol.

As for liveness, note that each slice is ε-successful except with probability $\mathcal{O}(t)\varepsilon_{\mathsf{fail}}(\varepsilon, f_{\mathrm{I}}, f_{\mathrm{E}}, L)$, which follows from Lemma 1 and a union bound, and thus, that all pipelines are successful except for the same error probability due to Lemma 2. We thus make this assumption and account for the respective error.

To determine r', consider any time interval $I_\eta = [t_0, t_0 + \eta]$, by assumption guaranteeing the inclusion of an honest base-protocol payload (i.e. EB). We estimate how old a Leios payload (i.e. transaction) can be that gets referenced by this EB.

The pipeline EB belongs to, must have started at time $t_0 - t_{\mathsf{PL}} - L$ or later, where L accounts for the fact that (by construction) FtchPld delivers an EB of the most recent succesful pipeline which finished at most L before. Similarly, as the r' period may start out of sync with the pipeline slices, we need to account for another time slack of L—as our IB-production guarantees are stated with respect to complete slices.

We thus conclude that I_η guarantees the inclusion of a Leios payload that was fetched no earlier than $t_0 - t_{\mathsf{PL}} - 2L$. We thus get $r' = 2L + t_{\mathsf{PL}} + \eta$, and respective base-chain settlement within $2L + t_{\mathsf{PL}} + u$. Finally, by Lemma 3, all IBs referenced by such EB will be fully available νL after the Deliver2 stage, yielding the given u'. □

Obviously, the guarantee given by Theorem 1 is quite weak as it is only meaningful for large enough f_{I} and large enough L due to the union bound over t. Indeed, if the failure probability of the pipelines is too large, then, depending on the underlying (black-box) protocol, it remains unclear with what probability honest base-protocol payloads can be guaranteed to include an EB from a successful pipeline—an uncertainty that is eliminated in Full Leios.

Theorem 2. *Let ε be such that $0 < \varepsilon < 2\alpha_{\mathrm{H}} - 1$, and assume the base protocol Π_{base} achieves*

- *persistence with parameter w, and*
- *ledger quality with parameters η and u.*

Then, Simple Leios achieves $(\varepsilon_{\mathsf{tp}}(t), R)$-throughput for

$$\varepsilon_{\mathsf{tp}}(t) = \mathcal{O}(t)\varepsilon_{\mathsf{fail}}(\varepsilon, f_{\mathrm{I}} f_{\mathrm{E}}, L) + \varepsilon_{\mathsf{base}}(t)$$

and

$$R \;=\; \frac{1-\varepsilon}{1+\varepsilon}\frac{\lambda - \beta'}{\lambda + 1}\min(L/\eta, 1) \cdot \alpha_{\mathrm{H}} \cdot C \;.$$

Proof (of Theorem 2). Consider an arbitrary $\gamma > 0$ (as required by Definition 5) and fix an arbitrary time t. In order to establish the theorem, one needs to show that if $t \geq t_0$ (for a t_0 to be determined), then $|\mathcal{L}_{\mathsf{san}}^{(t)}|/t \geq (1-\gamma)R$ (where $\mathcal{L}_{\mathsf{san}}^{(t)}$ is the sanitized ledger at time t).

In the following, assume that, up to time t, all pipelines are ε-successful (for ε as in the theorem statement) and that there are no errors in the base protocol; this fails to occur with probability $\mathcal{O}(t)\varepsilon_{\mathsf{fail}}(\varepsilon, f_{\mathrm{I}} f_{\mathrm{E}}, L) + \varepsilon_{\mathsf{base}}(t) = \varepsilon_{\mathsf{tp}}(t)$.

The first step is to determine a lower bound on the number of pipelines that are referenced in the ledger of the base protocol by time t. To that end, let k be maximal such that three consecutive intervals of length $t_P := (5 + \mu + \lambda)L$, kL, and $\max(\eta + u, \nu L)$, respectively, fit into t. Observe that k pipelines end in the middle interval (of length kL). Since the ledger quality (cf. Definition 4) of the base ledger guarantees that an honest payload is included in the base ledger every η slots, at least

$$\min(kL/\eta, k) \;=\; k \min(L/\eta, 1)$$

pipelines from the middle interval are referenced in the base ledger at time t. Further, all corresponding input blocks from these pipelines are globally delivered by time t (the raison d'être for the third interval) and will therefore be reflected in $\mathcal{L}_{\mathsf{san}}^{(t)}$.

The second step is to observe that each (ε-successful pipeline) references at least $(1-\varepsilon)\alpha_{\mathrm{H}} f_{\mathrm{I}} L$ input blocks. Hence, the total amount of data appearing in $\mathcal{L}_{\mathsf{san}}^{(t)}$ is at least

$$k \min(L/\eta, 1) \cdot (1-\varepsilon)\alpha_{\mathrm{H}} f_{\mathrm{I}} L B \;=\; kL \cdot R\,,$$

for the choice of f_{I} as in (1).

Consider now

$$\frac{|\mathcal{L}_{\mathsf{san}}^{(t)}|}{t} \;\geq\; \frac{kL}{t} R \;\geq\; \frac{t - t_P - \max(\eta + u, \nu L)}{t} R\,.$$

Thus, one may pick t_0 such that

$$\frac{t_0 - t_P - \max(a\eta + u, \nu L)}{t_0} \;\geq\; (1-\gamma)\,.$$

□

In practice, some of the overall bandwidth $\overline{C}$ needs to be allocated to the base protocol and to the diffusion of EBs and votes. However, observe that the larger $\overline{C}$ is, the smaller the fraction of bandwidth taken up by those parts becomes as the amount of traffic produced by them does not increase as one increases the IB data rate (by increasing the size of IBs). Hence, one get arbitrarily close to throughput $\min(L/\eta, 1)\alpha_{\mathrm{H}}\overline{C}$, for large enough $\overline{C}$. Specifically, one proves the following asymptotic result:

Corollary 1. *For any δ, $0 < \delta < \Theta(\beta/L)$, Simple Leios achieves a throughput of $(1-\delta)\min(L/\eta, 1)\alpha_{\mathrm{H}}\overline{C}$ at the cost of a latency of $\mathcal{O}(\beta\delta^{-3})$, for large enough* overall *network capacity $\overline{C}$.*

Proof. Let ℓ be the total bandwidth allocated to EB and vote diffusion as well as to running the base protocol, i.e., $\overline{C} = C + \ell$, where C is (as usual) the capacity allocated to diffusing input blocks and ℓ is the capacity reserved for EBs, votes, and the base protocol. Using Theorem 2, one obtains that Simple Leios achieves throughput

$$\frac{1-\varepsilon}{1+\varepsilon}\frac{\lambda - \beta'}{\lambda + 1} \min(L/\eta, 1)\alpha_{\mathrm{H}} C \;=\; \frac{1-\varepsilon}{1+\varepsilon}\frac{\lambda - \beta'}{\lambda + 1}\left(1 - \frac{\ell}{\overline{C}}\right) \min(L/\eta, 1)\alpha_{\mathrm{H}}\overline{C}\,.$$

For a given δ, set $\varepsilon := \delta/6$ and

$$\lambda \;:=\; \frac{3(\beta' + 1)}{\delta} - 1 \;=\; \mathcal{O}(\beta'/\delta)\,,$$

where $\lambda \geq 1$ requires $\delta < 3/2 \cdot (\beta' + 1) = \Theta(\beta/L)$. Then, for $\overline{C} \geq 3\ell/\delta$, one gets that each of the three factors in front of $\min(L/\eta, 1)\alpha_{\mathrm{H}}\overline{C}$ is at least $1 - \delta/3$. Using that $(1-\delta/3)^3 \geq 1 - \delta$, the first part of the corollary follows.

As for latency, observe that by Lemma 3 and Theorem 1, the latency of Simple Leios is bounded by $\mathcal{O}(\eta + u + \lambda^3 L/\beta'^2)$. For the above choice of λ, one obtains

$$\lambda^3 L/\beta'^2 \;=\; \mathcal{O}\left(\frac{\beta'^3}{\delta^3}\frac{1}{\beta'^2}L\right) \;=\; \mathcal{O}(\beta' L/\delta^3) \;=\; \mathcal{O}(\beta/\delta^3)\,,$$

which dominates as $\delta \to 0$. □

Observe that the throughput/latency tradeoff only works for $\delta < \Theta(\beta/L)$. As pointed out in Section 1.2, since Simple Leios has security error at least $e^{-\Omega(L)}$, L must be chosen at least superlogarithmic in the security parameter, which means that the tradeoff is only meaningful in cases where the resulting latency $\mathcal{O}(\beta/\delta^{-3})$ is already superlogarithmic. In particular, if one is only interested in, say, $\delta = 1/2$, then the latency will behave as $\mathcal{O}(L/\beta'^2) = \mathcal{O}(L^3/\beta^2)$, which is again superlogarithmic in the security parameter. Full Leios (cf. Section 5) will not suffer from this limitation, as there the security error is independent of L, which allows to choose L independently of the security parameter.

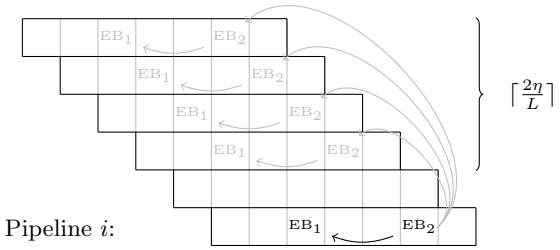

Fig. 6. Augmented endorsing pipeline. A new pipeline starts every L slots, and from the roughly $2\eta/L$ previous pipelines, all successful ones must be covered to guarantee full EB_2 coverage despite poor quality η of Π_{base}.

5 Full Leios

5.1 The Protocol

The objective of Full Leios is to maintain high throughput independent of the ledger quality η of the base protocol. This is accomplished by modifying Simple Leios (cf. Section 4) such that pipelines reference each other for a limited time horizon proportional to η, thus facilitating indirect ledger inclusion of all successful pipelines that emerged after the last honest payload entered the ledger.

This recursion is implemented by adapting the EB list $L_{\mathsf{E}} = (L_{\mathsf{E},1}, L_{\mathsf{E},2})$ so that it consists of two sublists $L_{\mathsf{E},1}$ and $L_{\mathsf{E},2}$, where $L_{\mathsf{E},1}$ takes on the role of L_{E} in Simple Leios, and $L_{\mathsf{E},2}$ references EBs from previous pipelines. Concretely, the rules for the ENDORSE stage are adapted as follows (cf. Figure 6), where i is the index of the pipeline instance:

1. ENDORSE: endorser blocks from this stage reference
   (a) in $L_{\mathsf{E},1}$, all VOTE1-certified EBs from the LINK stage (as before), and
   (b) in $L_{\mathsf{E},2}$, for each of the pipeline instances $i - \lceil 2\eta/L \rceil, \ldots, i-2$, one VOTE2-certified EB from its ENDORSE stage—if such an EB exists.

Note that pipeline recursion by EB skips pipeline instance $i-1$. The reason for this is that re-inclusion needs to wait for VOTE2-certification of prior endorser blocks, which for the case of the immediate predecessor pipeline, happens during the same slice wherein said EB is being produced.

A single base-protocol payload can now include multiple pipelines by containing one single EB together with its certificate. Still, two problems need to be solved to guarantee complete ledger inclusion of all successful pipelines:

Issue 1. The EB recursion skips one instance. Thus, in case of direct ledger inclusion of an EB of pipeline instance i, we need to make sure that the recursion span is large enough for the next EB in an honest payload to catch up on the inclusion of pipeline instance $i-1$.

Issue 2. The adversary may craft EBs that omit successful pipelines they are supposed to cover in order to censor their IB production. We thus need a mechanism to detect such behavior such that honest base-protocol inclusion of such EBs is avoided.

Concrete rules. We modify the VOTE2-stage checks and define a rule for direct ledger inclusion of an EB. Together, these changes address the two problems stated at the end of the previous paragraph.

Specifically, besides the VOTE2-stage checks performed in Simple Leios (see Figure 5), an EB (assume it to belong to pipeline instance i) must satisfy the following property with respect to the voting party's view:

– for each index $j \in \{i - \lceil 2\eta/L \rceil, \ldots, i-2\}$, if a VOTE2-certified EB_j of that pipeline instance was locally seen Δ_{hdr} before the last slot of pipeline j, then EB must reference, in $L_{\mathsf{E},2}$, a VOTE2-certified EB'_j of the same pipeline instance.

Finally, recall the validity predicate for the base ledger V_{chkCerts} enforces that an EB is eligible for inclusion in the base ledger iff it satisfies the following properties:

- it is VOTE2-certified, and,
- no EB from the same pipeline instance i is *directly* included in the most recent base-protocol payload (last EB in $\tilde{\mathcal{L}}$ as returned by (FTCHLDG)).

The second EB eligibility property together with EB recursion span 2η address Issue 1, as becomes evident in the analysis below. The extra VOTE2-check addresses Issue 2—by enabling the honest payload producer to detect censorship.

5.2 Analysis

In the following, let t be the time the protocol is run and let $\varepsilon_{\mathsf{base}}(t)$ be the error of the base protocol when run for time t. Recall that $n_{\mathsf{PL}} = 5 + \lambda + \mu$ denotes the number of slices per pipeline, and $t_{\mathsf{PL}} = n_{\mathsf{PL}} L$ is the length of a pipeline in slots.

Security. Full Leios inherits its persistence from the base protocol. Its liveness rests on the fact that all successful pipelines are ultimately referenced via EB inclusion in the base ledger. Specifically, one proves the following lemma:

Lemma 4. *Given that there are no base-protocol errors, all successful pipelines are ultimately covered from the base protocol. In particular, a successful pipeline, i.e., its referenced IBs, settles in the base-protocol within $\eta + u$ time after pipeline completion.*

Proof. We first argue that an honest EB of a successful pipeline is eligible for base-protocol inclusion, i.e, it is VOTE2-certified. Now, consider an honest EB candidate for VOTE2-stage voting by an honest party P. Any previous pipeline that P requires to be referenced by EB (as defined by the new voting rule) will indeed be referenced by EB as the respective certificate from P's view was available to the producer of EB by the end of that pipeline. Thus every honest EB will be VOTE2-certified and will be thus be eligible for base-protocol inclusion.

We now show that all successful pipelines will eventually be referenced in the ledger. For this, consider any successful pipeline instance i ending at time slot t_i (and producing an honest VOTE2-certified EB_i), and assume that an EB of that pipeline does not get *directly* included as a ledger payload (otherwise, we are done).

Furthermore, let t be such that an honest payload $p \in \boldsymbol{p}_t$ becomes part of the ledger. Ledger quality guarantees that $t \leq t_i + \eta$. We distinguish two cases:

1. p contains an EB_j from a pipeline instance $j > i + 1$, or,
2. p contains an EB_j from pipeline instance $j = i + 1$.

If $j > i + 1$ then EB_j references EB_i, as the recursion spans 2η. For the case $j = i + 1$, consider the next honest payload p' that enters the ledger after p, which, due to ledger quality, happens during time interval $[t, t + \eta] \subseteq [t_i, t_i + 2\eta]$. Since p' neither directly includes EB_i nor EB_j (by assumption and the predicate V_{chkCerts}), it directly includes an EB from a pipeline instance $k \in [i + 2, i + \lceil 2\eta/L \rceil]$, indirectly covering EB_i because of EB recursion span 2η.

Finally, ledger-quality parameter u implies that the base-protocol payload (i.e., EB) settles within $\eta + u$ after pipeline completion. □

As for the settlement delay for transactions, observe that a transaction settles once by the following four events have occurred, each of which is addressed individually:

(1) Synchronization with pipeline slices (as in the proof of Theorem 1): an additional delay of $2L$ is accounted for synchronizing the r' period with the slices, and the delay of base-protocol inclusion after pipeline completion.

(2) Payload inclusion in an honest IB of a successful pipeline and respective pipeline completion: A pipeline only fails if one if its $5 + \lambda + \mu = \mathcal{O}(\lambda^2/\beta')$ (recall that $\mu = \mathcal{O}(\lambda^2/\beta')$) fails. By a unoin bound, the probability a pipeline fails is thus at most $\varphi_{\mathsf{Pfail}} := (5 + \lambda + \mu)\varepsilon_{\mathsf{fail}} = \mathcal{O}(\lambda^2/\beta')\varepsilon_{\mathsf{fail}}$, where $\varepsilon_{\mathsf{fail}} := \varepsilon_{\mathsf{fail}}(\varepsilon, f_{\mathrm{I}}, f_{\mathrm{E}}, L)$ is the probability of a slice being unsuccessful (cf. Section 4.4), for an ε satisfying $0 < \varepsilon < 2\alpha_{\mathrm{H}} - 1$.

If considering adjacent but *non-overlapping pipelines*, the number of trials until a good pipeline is thus bounded by a geometric distribution with failure probability at most $\varphi_{\mathsf{Pfail}} < 1$ (assuming parameters are chosen accordingly). Hence, in expectation, the expected time to inclusion into a successful pipeline is upper bounded by $t_{\mathsf{PL}} \cdot (1 - \varphi_{\mathsf{Pfail}})^{-1} = \mathcal{O}(\lambda^2 L/\beta' \cdot (1 - \varphi_{\mathsf{Pfail}})^{-1})$.

(3) Base-protocol inclusion of an EB of that pipeline (directly or indirectly via a later pipeline) and settlement thereof: once Vote2-certified EBs are ready as payloads for the base protocol, by Lemma 4, such an EB will be included within $\eta + u$.

(4) A queue-clearing event happens after that pipeline has completed: in addition to a corresponding EB having to be included in the base ledger, it is also necessary for *all* input blocks referenced by EBs included before EB do be delivered.

This is modeled by the "queue clearing" game defined in Appendix E. Specifically, one bounds the expected time $\mathbb{E}[T_{\mathsf{clear}}]$ for the so-called *timely IBs* in $\mathcal{F}_{\mathsf{diff}}$'s "buffer" at a particular time $\tilde{t}$ to be delivered to all parties. Timely IBs are IBs released into the network before they reach a certain age A_{max}.

Specifically, consider an IB from some slot s. For the purposes of using Theorem 7 for Full Leios, one sets

- $Np := f_{\mathrm{I}}$ (one may pick N and p arbitrarily subject to this constraint),
- $A_{\mathsf{max}} := (2 + \lambda + \mu)L$ because an IB has to be introduced into the network before Vote1 in order to ever be referenced by any Vote2-certified EB,
- $\tilde{t}$ to the first slot after the end of the pipeline containing s.

Applying the theorem thus yields,

$$\mathbb{E}[T_{\mathsf{clear}}] \;\leq\; \frac{2\gamma^2}{(1-\tau Np)^2} + \frac{144\tau^2 Np^2}{(1-\tau Np)^4} \;,$$

for $\gamma = A_{\mathsf{max}} + \beta = (2+\lambda+\mu)L + \beta = \mathcal{O}(\lambda^2 L/\beta' + \beta)$.[25] Using that $\tau Np = B/Cf_{\mathrm{I}} = \mathcal{O}(\lambda/(\lambda+1))$—based on the choice $f_{\mathrm{I}} = \mathcal{O}(\lambda/(\lambda+1) \cdot C/B)$—one obtains that

$$\mathbb{E}[T_{\mathsf{clear}}] \;=\; \mathcal{O}(\lambda^6 L^2/\beta'^2 + \beta^2) \;.$$

Combining the above, one gets settlement in expected time

$$2L + t_{\mathsf{PL}} \cdot (1 - \varphi_{\mathsf{Pfail}})^{-1} + \eta + u + \mathbb{E}[T_{\mathsf{clear}}] \;=\; 2L + \eta + u + \mathcal{O}\left(\frac{\lambda^2 L}{\beta'}(1 - \varphi_{\mathsf{Pfail}})^{-1} + \frac{\lambda^6 L^2}{\beta'^2} + \beta^2\right) .$$

This yields:

Theorem 3. *Let ε be such that $0 < \varepsilon < 2\alpha_{\mathrm{H}} - 1$, and assume the base protocol Π_{base} achieves*

- *persistence with parameter w and*
- *quality with parameters η and u.*

Then, Full Leios achieves

- *persistence with parameter $w' = w$, and*
- *liveness with parameters r' and u' for $\mathbb{E}[r'] = \mathbb{E}[u'] = 2L+\eta+u+\mathcal{O}\left(\lambda^2 L/\beta'(1-\varphi_{\mathsf{Pfail}})^{-1} + \lambda^6 L^2/\beta'^2 + \beta^2\right)$,*

except with probability $\varepsilon_{\mathsf{base}}(t)$.

[25] Recall that β' is such that $\beta = \beta' L$ and that $\mu = \mathcal{O}(\lambda^2/\beta')$.

Throughput. The throughput analysis in Full Leios is based on the following key insights: (1) the expected fraction of unsuccessful pipelines is at most φ_{Pfail} (see above), and (2) all successful pipelines are covered by the recursive inclusion.

Theorem 4. *Let ε be such that $0 < \varepsilon < 2\alpha_{\mathrm{H}} - 1$ and let κ be a security parameter satisfying $\kappa \geq \mathcal{O}(\lambda^3 L^2/\beta + \beta\lambda)$. Assume the base protocol Π_{base} achieves*

- *persistence with parameter w, and*
- *ledger quality with parameters η and u.*

Then, Full Leios achieves $(\varepsilon_{\mathsf{tp}}(t), R)$-throughput for

$$\varepsilon_{\mathsf{tp}}(t) \;=\; \varepsilon_{\mathsf{base}}(t) + e^{-\Omega(\kappa)} + \mathcal{O}(\lambda^2) \cdot e^{-\Omega(\lambda^{-2}\kappa)}$$

and

$$R \;=\; \frac{1-\varepsilon}{1+\varepsilon}\frac{\lambda - \beta'}{\lambda + 1}(1 - \varphi_{\mathsf{Pfail}})\alpha_{\mathrm{H}} C\,.$$

Proof. Consider an arbitrary $\gamma > 0$ (as required by Definition 5) and fix an arbitrary time t. In order to establish the theorem, one needs to show that if $t \geq t_0$ (for a t_0 to be determined), then $|\mathcal{L}_{\mathsf{san}}^{(t)}|/t \geq (1-\gamma)R$ (where $\mathcal{L}_{\mathsf{san}}^{(t)}$ is the sanitized ledger at time t), except with error $\varepsilon_{\mathsf{tp}}(t)$.

In the following, assume that, up to time t, there are no errors in the base protocol; this fails to occur with probability $\varepsilon_{\mathsf{base}}(t)$.

The first step is to determine a lower bound on the number of pipelines that are referenced in the ledger of the base protocol by time t. To that end, let k be maximal such that three consecutive intervals of length $t_P := (5 + \lambda + \mu)L$, kL, and $\max(\eta + u, \kappa)$, respectively, fit into t. Observe that k pipelines end in the middle interval (of length kL). The idea is now to show that with high probability, (I) the fraction of unsuccessful pipelines among these is close to φ_{Pfail}, and (II) all input blocks belonging to those pipelines are delivered by the end of the third interval. Combining this with Lemma 4, stating that all successful pipelines are referenced in the base ledger yields the theorem.

Consider (I) first. Since slice successes are independent, by Chernoff bound, the number of ε-unsuccessful slices in the first two intervals is at most

$$(1 + \varepsilon')\varepsilon_{\mathsf{fail}} \cdot (5 + \lambda + \mu + k)$$

with probability at least $1 - e^{-\varepsilon'^2 \Omega(k)}$, for any constant $\varepsilon' > 0$. Correspondingly, the number of ε-unsuccessful pipelines is at most

$$(1 + \varepsilon')\varepsilon_{\mathsf{fail}} \cdot (5 + \lambda + \mu + k)\mathcal{O}(\lambda^2/\beta') \;\overset{!}{\leq}\; \varphi_{\mathsf{Pfail}} \cdot k$$

since each slice can affect at most $\mathcal{O}(\lambda^2/\beta')$ pipelines; the equality requires that $k \geq \Theta(\mu) = \Theta(\lambda^2/\beta')$, which implies that one needs

$$t \;\geq\; kL \;=\; \mathcal{O}(\lambda^2 L/\beta')\,. \tag{3}$$

Consequently, there are at least $(1 - \varphi_{\mathsf{Pfail}}) \cdot k$ ε-successful slices.

Consider now (II). This follows from the queue-clearing argument in Appendix E. Specifically, let $\tilde{t}$ be the last slot of the middle interval. Then, by Lemma 9, with parameters as in the above security analysis (Theorem 3), one obtains that the probability the queue is not cleared within $\max(\eta + u, \kappa)$ slots, is at most

$$\exp\left(-\frac{(\kappa - 1)Np\delta_0^2}{3}\right) \cdot \frac{3}{Np\delta_0^2} \;=\; \exp\left(-\Omega\left(\frac{\kappa(1 - \tau Np)^2}{\tau^2 Np}\right)\right) \cdot \mathcal{O}\left(\frac{\tau^2 Np}{(1 - \tau Np)^2}\right),$$

where the equality follows from Lemma 10; the condition $\kappa \geq \mathcal{O}(\lambda^3 L^2/\beta + \beta\lambda)$ is required to satisfy the preconditions of Lemmas 9 and 10. Plugging in values for τ, N, p, and subsequently for f_{I} yields an error of

$$\mathcal{O}(\lambda^2) \cdot e^{-\Omega(\lambda^{-2}\kappa)}$$

for (II). Note that the above requires

$$t \;\geq\; \max(\eta+u,\kappa) \;=\; \max(\eta+u,\mathcal{O}(\lambda^3 L^2/\beta+\beta\lambda))\,. \tag{4}$$

The second step is to observe that each (ε-successful pipeline) references at least $(1-\varepsilon)\alpha_{\mathrm{H}} f_{\mathrm{I}} L$ input blocks. Hence, the total amount of data appearing in $\mathcal{L}^{(t)}_{\mathsf{san}}$ is at least

$$(1-\varphi_{\mathsf{Pfail}})k\cdot(1-\varepsilon)\alpha_{\mathrm{H}} f_{\mathrm{I}} L B \;=\; kL\cdot R\,,$$

for the choice of f_{I} as in (1).

Consider now

$$\frac{|\mathcal{L}^{(t)}_{\mathsf{san}}|}{t} \;\geq\; \frac{kL}{t}R \;\geq\; \frac{t-t_P-\max(\eta+u,\nu L)}{t}R\,.$$

Thus, one may pick t_0 that satisfies

$$\frac{t_0-t_P-\max(\eta+u,\nu L)}{t_0} \;\geq\; (1-\gamma)\,.$$

as well as (3) and (4). □

In practice, some of the overall bandwidth $\overline{C}$ needs to be allocated to the base protocol and to the diffusion of EBs and votes. However, observe that the larger $\overline{C}$ is, the smaller the fraction of bandwidth taken up by those parts becomes as the amount of traffic produced by them does not increase as one increases the IB data rate (by increasing the size of IBs). Hence, one gets arbitrarily close to throughput $\alpha_{\mathrm{H}}\overline{C}$, for large enough $\overline{C}$. Specifically, one proves the following asymptotic result:

Corollary 2. *For any δ, $0<\delta<\Theta(\beta/L)$, Full Leios achieves a throughput of $(1-\delta)\alpha_{\mathrm{H}}\overline{C}$ at the cost of an expected latency of $\mathcal{O}(\beta\delta^{-2})$, for large enough* overall *network capacity $\overline{C}$.*

Proof. Let $\overline{C}=C+\ell$, where C is (as usual) the capacity allocated to diffusing input blocks and ℓ is the capacity reserved for EBs, votes, and the base protocol. Using Theorem 4, one obtains that Full Leios achieves throughput

$$\frac{1-\varepsilon}{1+\varepsilon}\frac{\lambda-\beta'}{\lambda+1}(1-\varphi_{\mathsf{Pfail}})\alpha_{\mathrm{H}}C \;=\; \frac{1-\varepsilon}{1+\varepsilon}\frac{\lambda-\beta'}{\lambda+1}\left(1-\frac{\ell}{\overline{C}}\right)(1-\varphi_{\mathsf{Pfail}})\alpha_{\mathrm{H}}\overline{C}\;.$$

For a given δ, set $\varepsilon:=\delta/8$;

$$\lambda \;:=\; \frac{4(\beta'+1)}{\delta}-1 \;=\; \mathcal{O}(\beta'/\delta)\;,$$

where $\lambda\geq 1$ requires $\delta<2(\beta'+1)=\Theta(\beta/L)$; and

$$L \;=\; \mathcal{O}\left(\frac{\beta^{3/2}}{\delta^2}\log\left(\frac{\beta}{\delta}\right)\right)\;.$$

Then, for $\overline{C}\geq 4\ell/\delta$, one gets that each of the four factors in front of $\alpha_{\mathrm{H}}\overline{C}$ is at least $1-\delta/4$. Using that $(1-\delta/4)^4\geq 1-\delta$, the first part of the corollary follows.

As for latency, observe that by Theorem 3, the expected latency of Full Leios is bounded by

$$\mathcal{O}(a\cdot\lambda^2 L/\beta'+\max(\eta+u,\lambda^6 L^2/\beta'^2+\beta^2))\;,$$

where $a = (1 - \varphi_{\mathsf{Pfail}})^{-1}$. For the above choices of λ and L, one obtains

$$\begin{aligned}
\mathcal{O}\left(a\lambda^2 L/\beta' + \max\left(\eta + u,\ \lambda^6 L^2/\beta'^2 + \beta^2\right)\right) &= \mathcal{O}\left(a\frac{\lambda^2 L}{\beta'}\right) + \mathcal{O}\left(\max\left(\eta + u,\ \frac{\lambda^6 L^2}{\beta'^2} + \beta^2\right)\right) \\
&= \mathcal{O}\left(a\frac{(\beta'/\delta)^2 L}{\beta'}\right) + \mathcal{O}\left(\max\left(\eta + u,\ \frac{(\beta'/\delta)^6 L^2}{\beta'^2} + \beta^2\right)\right) \\
&= \mathcal{O}\left(a\frac{\beta'}{\delta^2} L\right) + \mathcal{O}\left(\max\left(\eta + u,\ \frac{\beta'^4}{\delta^6} L^2 + \beta^2\right)\right) \\
&= \mathcal{O}\left(a\frac{\beta}{\delta^2}\right) + \mathcal{O}\left(\max\left(\eta + u,\ \frac{\beta^4}{\delta^6} \cdot \frac{1}{L^2} + \beta^2\right)\right) \\
&= \mathcal{O}\left(a\frac{\beta}{\delta^2}\right) + \mathcal{O}\left(\max\left(\eta + u,\ \frac{\beta}{\delta^2 \log^2(\beta/\delta)} + \beta^2\right)\right) .
\end{aligned}$$

Note that—given that parameters were chosen such that $\varphi_{\mathsf{Pfail}} < \delta/4$—one has $a = \Theta(1)$ for $\delta \to 0$. Thus, considering η and u imposed by the base protocol as constants, for $\delta \to 0$, the above expression is dominated by $\mathcal{O}(\beta/\delta^2)$. □

6 Equivocation Handling

There are three types of protocol messages in Leios: input blocks (IBs), endorser blocks (EBs), and votes. All of these messages are diffused via $\mathcal{F}_{\mathsf{diff}}$, which means that by introducing equivocations, the adversary can split the views of honest parties in terms of which their preferred message is for a given lottery ID. Therefore, for each message type, such *splits* must be dealt with by an explicit strategy.

This section presents a strategy that guarantees that for each block/vote opportunity in the Leios overlay protocol:

- each honest party downloads and forwards *at most two block headers*,[26]
- each honest party downloads *at most one block body*, and
- only the fully downloaded block/vote is required for the protocol (and, in particular, ledger correctness).

The strategy presented below assumes that the Link stage does not start until $5\Delta_{\mathsf{hdr}}$ after the last IB production opportunity in the Propose stage, which is the case if $\lambda L \geq 5\Delta_{\mathsf{hdr}}$.

Input blocks. Observe that for each block opportunity, $\mathcal{F}_{\mathsf{diff}}$ delivers to a party P at most one body, namely the body belonging to the first header received by P;[27] below, the actual block downloaded by P is called the *preferred block for* P. First, one has to ensure that EBs are only voted for if for *all* referenced IBs $\mathsf{IB} = (h, b)$, all honest parties have the same preferred IB for $\mathsf{LID}(h)$. This is achieved by adding an additional check to the Vote1-rule:

- A voter P in the Vote1 stage only votes for an EB if *all* referenced IBs $\mathsf{IB} = (h, b)$ satisfy: P received (a) h before time $s + 2\Delta_{\mathsf{hdr}}$ and (b) no equivocating header h' before time $s + 4\Delta_{\mathsf{hdr}}$, where s is the slot number of h.

The $2\Delta_{\mathsf{hdr}}$ gap between criterions (a) and (b) ensure that no honest party P' has a preferred header different from P: Assume some P' preferes a different header h'. Then, P' would have to have received h' before $s + 3\Delta_{\mathsf{hdr}}$ as h or a proof of equivocation reach P' at $s + 3\Delta_{\mathsf{hdr}}$. In either case, P would see a proof of

[26] Recall that due to their small size, EBs and votes are modeled as "header-only," i.e., they have an empty body, whereas an IB consists of a header and a body, where the body contains the actual payload. Downloading a *bounded* number of headers is acceptable since they are rather small compared to bodies.

[27] As explained in Section 3, this behavior is straightforward to implement in the underlying gossip protocol.

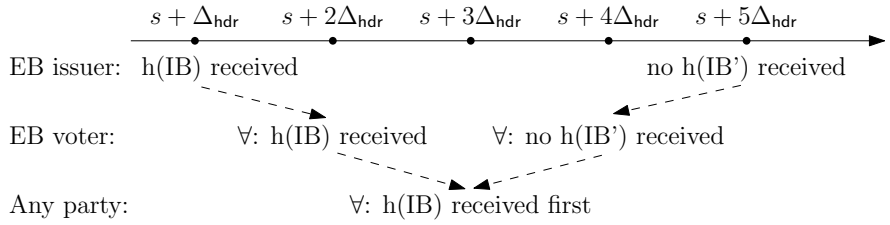

Fig. 7. Equivocation handling for input blocks. h(IB) stands for "header of IB."

equivocation before $s + 4\Delta_{\mathsf{hdr}}$. Hence, an EB is only voted for by honest parties if there is no split for any IBs it references.

It remains to determine, for EB producers in the Link stage, an IB-inclusion strategy that guarantees that their EBs are voted for based on the above voting rule:

– An EB producer P in the Link-stage only references $\mathsf{IB} = (h, b)$ in list L_{I} if P received (i) h before time $s + \Delta_{\mathsf{hdr}}$, and (ii) no equivocating header h' before time $s + 5\Delta_{\mathsf{hdr}}$.

Criterion (i) guarantees that criterion (a) is satisfied for the voters, and (ii) guarantees that (b) is satisfied for the voters. Hence, any honest EB from the Link-stage will be voted for in the Vote1 stage. The IB equivocation strategy is illustrated in Figure 7.

Endorser blocks (EB). Along the above lines, we need to ensure that a particular EB only becomes Vote2-certified if EB is the preferred block for its respective block opportunity. Let $\mathsf{EB} = (h)$ be an EB from some slot s; recall that EBs consist of headers only. The timing rule is as follows: a voter P in the vote stage only votes for an EB if P received (a) h before time $s + \Delta_{\mathsf{hdr}}$ and (b) no equivocating header h' before time $s + 3\Delta_{\mathsf{hdr}}$. Note, that if any honest party voted in favor of a block, then it must be the case that this block is the preferred one for all honest parties.

Votes. Note that votes for a particular EB are deterministic, and cannot be equivocated. Still, for a given voting opportunity, the adversary may create votes for different (equivocated) blocks of the same block opportunity. This problem can be easily mitigated by having parties only download (and diffuse) the votes corresponding to the preferred EB of a given block opportunity.

Remark 1. The equivocation handling mechanism does not inhibit late-joiners from selecting the correct chain, as it only affects the voting/certificate generation process. That is, as long as a valid certificate is generated, it is going to be valid for all parties, independently of the time they joined. This general principle allows late-joining nodes to unambiguously select the correct chain (which only involves certified EBs). However, it should be noted that late-joining nodes take some time to fully participate in the protocol, as voting for certain EBs requires knowledge of earlier arrival times that they do not have. Specifically, late-joining nodes should only actively participate in the protocol after having been fully synced for 2η slots.

Analysis. We prove some simple facts about the above mechanism that can be used in a straight-forward way to extend the security theorems of Sections 4 and 5 to the setting where block equivocations are considered.

Lemma 5. *For each block opportunity, each honest party downloads at most two block headers and one block body.*

Proof. The lemma directly follows by construction. □

Lemma 6. *If* EB *is voted for by an honest party, then all IBs referenced in* EB *are downloaded by all honest parties.*

Proof. See Fig. 7. An honest party only votes for EB if the header of each referenced IB was received by time $s + 2\Delta_{\mathsf{hdr}}$ where s is the slot of IB, and no equivocating header for any such IBs was received by time $s + 4\Delta_{\mathsf{hdr}}$ (middle row). This implies that, for each referenced IB, all honest parties received the header by time $s + 3\Delta_{\mathsf{hdr}}$ as the first header seen for the respective block opportunity (bottom row). All honest parties thus 'prefer' and download all IBs referenced in EB. □

Lemma 7. *If* EB *is voted for by an honest party, then* EB *gets downloaded by all honest parties.*

Proof. An honest party only votes for EB if the header of EB was received by time $s + \Delta_{\mathsf{hdr}}$ where s is the slot of EB, and no equivocating header for EB was received by time $s + 3\Delta_{\mathsf{hdr}}$. This implies that all honest parties received the header of EB by time $s + 2\Delta_{\mathsf{hdr}}$ as the first header for the given block opportunity, and thus download EB. □

Lemma 8. *Given that the respective pipeline is successful, every* EB *produced by an honest party gets voted for by each honest party.*

Proof. The IB timing rule for IB inclusion maintains enough slack to imply the necessary timing (see Fig. 7) for respective voting for every honest party. Similarly, the EB timing rule for voting is implied by the delivery guarantees for EB headers. □

Acknowledgements

Alexander Russell is both a Senior Research Fellow at IOG and the recipient of a research grant from IOG at the University of Connecticut.

References

1. Al-Bassam, M., Sonnino, A., Buterin, V.: Fraud and data availability proofs: Maximising light client security and scaling blockchains with dishonest majorities. arXiv preprint arXiv:1809.09044 (2018)
2. Avarikioti, G., Käppeli, L., Wang, Y., Wattenhofer, R.: Bitcoin security under temporary dishonest majority. In: Goldberg, I., Moore, T. (eds.) FC 2019. LNCS, vol. 11598, pp. 466–483. Springer, Cham (Feb 2019). https://doi.org/10.1007/978-3-030-32101-7_28
3. Avarikioti, Z., Heimbach, L., Schmid, R., Vanbever, L., Wattenhofer, R., Wintermeyer, P.: Fnf-BFT: Exploring performance limits of bft protocols (2020)
4. Badertscher, C., Gaži, P., Kiayias, A., Russell, A., Zikas, V.: Consensus redux: Distributed ledgers in the face of adversarial supremacy. In: 2024 IEEE 37th Computer Security Foundations Symposium (CSF). pp. 267–282. IEEE Computer Society, Los Alamitos, CA, USA (jul 2024). https://doi.org/10.1109/CSF61375.2024.00018, https://doi.ieeecomputersociety.org/10.1109/CSF61375.2024.00018
5. Badertscher, C., Ga?i, P., Kiayias, A., Russell, A., Zikas, V.: Consensus redux: Distributed ledgers in the face of adversarial supremacy. Cryptology ePrint Archive, Report 2020/1021 (2020), https://eprint.iacr.org/2020/1021
6. Badertscher, C., Gazi, P., Kiayias, A., Russell, A., Zikas, V.: Ouroboros genesis: Composable proof-of-stake blockchains with dynamic availability. In: Lie, D., Mannan, M., Backes, M., Wang, X. (eds.) ACM CCS 2018. pp. 913–930. ACM Press (Oct 2018). https://doi.org/10.1145/3243734.3243848
7. Bagaria, V.K., Kannan, S., Tse, D., Fanti, G.C., Viswanath, P.: Prism: Deconstructing the blockchain to approach physical limits. In: Cavallaro, L., Kinder, J., Wang, X., Katz, J. (eds.) ACM CCS 2019. pp. 585–602. ACM Press (Nov 2019). https://doi.org/10.1145/3319535.3363213
8. Buterin, V., Griffith, V.: Casper the friendly finality gadget. arXiv preprint arXiv:1710.09437 (2017)
9. Cachin, C., Tessaro, S.: Asynchronous verifiable information dispersal. In: 24th IEEE Symposium on Reliable Distributed Systems (SRDS'05). pp. 191–201. IEEE (2005)
10. Canetti, R.: Universally composable security: A new paradigm for cryptographic protocols. In: Proceedings 42nd IEEE Symposium on Foundations of Computer Science. pp. 136–145. IEEE (2001)
11. Castro, M., Liskov, B., et al.: Practical byzantine fault tolerance. In: OsDI. vol. 99, pp. 173–186 (1999)

12. Chakravarty, M.M.T., Chapman, J., MacKenzie, K., Melkonian, O., Jones, M.P., Wadler, P.: The extended UTXO model. In: Bernhard, M., Bracciali, A., Camp, L.J., Matsuo, S., Maurushat, A., Rønne, P.B., Sala, M. (eds.) FC 2020 Workshops. LNCS, vol. 12063, pp. 525–539. Springer, Cham (Feb 2020). https://doi.org/10.1007/978-3-030-54455-3_37
13. Croman, K., Decker, C., Eyal, I., Gencer, A.E., Juels, A., Kosba, A.E., Miller, A., Saxena, P., Shi, E., Sirer, E.G., Song, D., Wattenhofer, R.: On scaling decentralized blockchains - (A position paper). In: Clark, J., Meiklejohn, S., Ryan, P.Y.A., Wallach, D.S., Brenner, M., Rohloff, K. (eds.) FC 2016 Workshops. LNCS, vol. 9604, pp. 106–125. Springer, Berlin, Heidelberg (Feb 2016). https://doi.org/10.1007/978-3-662-53357-4_8
14. Danezis, G., Kokoris-Kogias, L., Sonnino, A., Spiegelman, A.: Narwhal and tusk: a dag-based mempool and efficient bft consensus. In: Proceedings of the Seventeenth European Conference on Computer Systems. pp. 34–50 (2022)
15. Danezis, G., Meiklejohn, S.: Centrally banked cryptocurrencies. In: NDSS 2016. The Internet Society (Feb 2016). https://doi.org/10.14722/ndss.2016.23187
16. David, B., Gazi, P., Kiayias, A., Russell, A.: Ouroboros praos: An adaptively-secure, semi-synchronous proof-of-stake blockchain. In: Nielsen, J.B., Rijmen, V. (eds.) EUROCRYPT 2018, Part II. LNCS, vol. 10821, pp. 66–98. Springer, Cham (Apr / May 2018). https://doi.org/10.1007/978-3-319-78375-8_3
17. Dembo, A., Kannan, S., Tas, E.N., Tse, D., Viswanath, P., Wang, X., Zeitouni, O.: Everything is a race and nakamoto always wins. In: Ligatti, J., Ou, X., Katz, J., Vigna, G. (eds.) ACM CCS 2020. pp. 859–878. ACM Press (Nov 2020). https://doi.org/10.1145/3372297.3417290
18. Dickerson, T.D., Gazzillo, P., Herlihy, M., Koskinen, E.: Adding concurrency to smart contracts. In: Schiller, E.M., Schwarzmann, A.A. (eds.) 36th ACM PODC. pp. 303–312. ACM (Jul 2017). https://doi.org/10.1145/3087801.3087835
19. Dubhashi, D.P., Panconesi, A.: Concentration of measure for the analysis of randomized algorithms. Cambridge University Press (2009)
20. Eyal, I., Gencer, A.E., Sirer, E.G., van Renesse, R.: Bitcoin-ng: A scalable blockchain protocol. In: Argyraki, K.J., Isaacs, R. (eds.) 13th USENIX Symposium on Networked Systems Design and Implementation, NSDI 2016, Santa Clara, CA, USA, March 16-18, 2016. pp. 45–59. USENIX Association (2016), https://www.usenix.org/conference/nsdi16/technical-sessions/presentation/eyal
21. Eyal, I., Sirer, E.G.: Majority is not enough: Bitcoin mining is vulnerable. In: Christin, N., Safavi-Naini, R. (eds.) FC 2014. LNCS, vol. 8437, pp. 436–454. Springer, Berlin, Heidelberg (Mar 2014). https://doi.org/10.1007/978-3-662-45472-5_28
22. Fitzi, M., Gaži, P., Kiayias, A., Russell, A.: Parallel chains: Improving throughput and latency of blockchain protocols via parallel composition. Cryptology ePrint Archive, Report 2018/1119 (2018), https://eprint.iacr.org/2018/1119
23. Fitzi, M., Wang, X., Kannan, S., Kiayias, A., Leonardos, N., Viswanath, P., Wang, G.: Minotaur: Multi-resource blockchain consensus. In: Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security. pp. 1095–1108 (2022)
24. Garay, J.A., Kiayias, A., Leonardos, N.: The bitcoin backbone protocol: Analysis and applications. In: Oswald, E., Fischlin, M. (eds.) EUROCRYPT 2015, Part II. LNCS, vol. 9057, pp. 281–310. Springer, Berlin, Heidelberg (Apr 2015). https://doi.org/10.1007/978-3-662-46803-6_10
25. Garay, J.A., Kiayias, A., Leonardos, N.: The bitcoin backbone protocol with chains of variable difficulty. In: Katz, J., Shacham, H. (eds.) CRYPTO 2017, Part I. LNCS, vol. 10401, pp. 291–323. Springer, Cham (Aug 2017). https://doi.org/10.1007/978-3-319-63688-7_10
26. Gazi, P., Kiayias, A., Russell, A.: Tight consistency bounds for bitcoin. In: Ligatti, J., Ou, X., Katz, J., Vigna, G. (eds.) ACM CCS 2020. pp. 819–838. ACM Press (Nov 2020). https://doi.org/10.1145/3372297.3423365
27. Gilad, Y., Hemo, R., Micali, S., Vlachos, G., Zeldovich, N.: Algorand: Scaling byzantine agreements for cryptocurrencies. In: Proceedings of the 26th symposium on operating systems principles. pp. 51–68 (2017)
28. Gupta, S., Hellings, J., Sadoghi, M.: Rcc: Resilient concurrent consensus for high-throughput secure transaction processing. arXiv preprint arXiv:1911.00837 (2019)
29. Kalodner, H.A., Goldfeder, S., Chen, X., Weinberg, S.M., Felten, E.W.: Arbitrum: Scalable, private smart contracts. In: Enck, W., Felt, A.P. (eds.) USENIX Security 2018. pp. 1353–1370. USENIX Association (Aug 2018)
30. Keidar, I., Kokoris-Kogias, E., Naor, O., Spiegelman, A.: All you need is DAG. In: Proceedings of the 2021 ACM Symposium on Principles of Distributed Computing. pp. 165–175 (2021)
31. Keidar, I., Naor, O., Poupko, O., Shapiro, E.: Cordial miners: Fast and efficient consensus for every eventuality. arXiv preprint arXiv:2205.09174 (2022)
32. Kiayias, A., Russell, A., David, B., Oliynykov, R.: Ouroboros: A provably secure proof-of-stake blockchain protocol. In: Katz, J., Shacham, H. (eds.) CRYPTO 2017, Part I. LNCS, vol. 10401, pp. 357–388. Springer, Cham (Aug 2017). https://doi.org/10.1007/978-3-319-63688-7_12

33. Kiffer, L., Neu, J., Sridhar, S., Zohar, A., Tse, D.: Security of nakamoto consensus under congestion. Cryptology ePrint Archive, Paper 2023/381 (2023), https://eprint.iacr.org/2023/381, https://eprint.iacr.org/2023/381
34. Lamport, L., Shostak, R., Pease, M.: The byzantine generals problem. ACM Transactions on Programming Languages and Systems (TOPLAS) **4**(3), 382–401 (1982)
35. Li, S., Yu, M., Avestimehr, A.S., Kannan, S., Viswanath, P.: PolyShard: Coded sharding achieves linearly scaling efficiency and security simultaneously. Cryptology ePrint Archive, Report 2018/925 (2018), https://eprint.iacr.org/2018/925
36. Liang, G., Vaidya, N.: Capacity of byzantine agreement with finite link capacity. In: 2011 Proceedings IEEE INFOCOM. pp. 739–747. IEEE (2011)
37. Micali, S., Rabin, M.O., Vadhan, S.P.: Verifiable random functions. In: 40th FOCS. pp. 120–130. IEEE Computer Society Press (Oct 1999). https://doi.org/10.1109/SFFCS.1999.814584
38. Milosevic, Z., Biely, M., Schiper, A.: Bounded delay in byzantine-tolerant state machine replication. In: 2013 IEEE 32nd International Symposium on Reliable Distributed Systems. pp. 61–70. IEEE (2013)
39. Nakamoto, S.: Bitcoin: A peer-to-peer electronic cash system. http://bitcoin.org/bitcoin.pdf (2008)
40. Nazirkhanova, K., Neu, J., Tse, D.: Information dispersal with provable retrievability for rollups. Cryptology ePrint Archive, Report 2021/1544 (2021), https://eprint.iacr.org/2021/1544
41. Neu, J., Sridhar, S., Yang, L., Tse, D., Alizadeh, M.: Longest chain consensus under bandwidth constraint. In: Proceedings of the 4th ACM Conference on Advances in Financial Technologies. pp. 126–147 (2022)
42. Pass, R., Seeman, L., shelat, a.: Analysis of the blockchain protocol in asynchronous networks. In: Coron, J.S., Nielsen, J.B. (eds.) EUROCRYPT 2017, Part II. LNCS, vol. 10211, pp. 643–673. Springer, Cham (Apr / May 2017). https://doi.org/10.1007/978-3-319-56614-6_22
43. Pass, R., Shi, E.: FruitChains: A fair blockchain. In: Schiller, E.M., Schwarzmann, A.A. (eds.) 36th ACM PODC. pp. 315–324. ACM (Jul 2017). https://doi.org/10.1145/3087801.3087809
44. Pass, R., Shi, E.: Rethinking large-scale consensus. In: Köpf, B., Chong, S. (eds.) CSF 2017 Computer Security Foundations Symposium. pp. 115–129. IEEE Computer Society Press (2017). https://doi.org/10.1109/CSF.2017.37
45. Pass, R., Shi, E.: The sleepy model of consensus. In: Takagi, T., Peyrin, T. (eds.) ASIACRYPT 2017, Part II. LNCS, vol. 10625, pp. 380–409. Springer, Cham (Dec 2017). https://doi.org/10.1007/978-3-319-70697-9_14
46. Rabin, M.O.: Efficient dispersal of information for security, load balancing, and fault tolerance. Journal of the ACM (JACM) **36**(2), 335–348 (1989)
47. Spiegelman, A., Giridharan, N., Sonnino, A., Kokoris-Kogias, L.: Bullshark: Dag bft protocols made practical. In: Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security. pp. 2705–2718 (2022)
48. Stathakopoulou, C., Tudor, D., Pavlovic, M., Vukolić, M.: Mir-bft: Scalable and robust bft for decentralized networks. Journal of Systems Research **2**(1) (2022)
49. Yin, M., Malkhi, D., Reiter, M.K., Gueta, G.G., Abraham, I.: Hotstuff: Bft consensus with linearity and responsiveness. In: Proceedings of the 2019 ACM Symposium on Principles of Distributed Computing. pp. 347–356 (2019)
50. Yin, M., Malkhi, D., Reiter, M.K., Gueta, G.G., Abraham, I.: Hotstuff: Bft consensus with linearity and responsiveness. In: Proceedings of the 2019 ACM Symposium on Principles of Distributed Computing. pp. 347–356 (2019)
51. Yu, H., Nikolic, I., Hou, R., Saxena, P.: OHIE: Blockchain scaling made simple. In: 2020 IEEE Symposium on Security and Privacy. pp. 90–105. IEEE Computer Society Press (May 2020). https://doi.org/10.1109/SP40000.2020.00008
52. Zamani, M., Movahedi, M., Raykova, M.: RapidChain: Scaling blockchain via full sharding. In: Lie, D., Mannan, M., Backes, M., Wang, X. (eds.) ACM CCS 2018. pp. 931–948. ACM Press (Oct 2018). https://doi.org/10.1145/3243734.3243853
53. Zhang, R., Zhang, D., Wang, Q., Wu, S., Xie, J., Preneel, B.: NC-max: Breaking the security-performance tradeoff in nakamoto consensus. In: Proceedings 2022 Network and Distributed System Security Symposium. pp. 1–18. NDSS (2022)

A Additional Related Work

This section details additional related work.

More details on throughput limitations of conventional protocols. Low transaction throughput is inherent to "conventional" blockchain protocols (pure Nakamoto or BFT) since block production is strictly sequential, and a new block must reach the subsequent leader before yet another block can be published. This lag inevitably sacrifices throughput, since the network overall remains underutilized waiting for a single block to propagate to all nodes. To see this in some more detail, in Nakamoto consensus [39] average throughput is bounded by fB, where f is the expected number of blocks per second and B the size of a block in terms of transaction bytes. Based on the security analysis of the Bitcoin blockchain, [26,17], a tight bound for security is $f\Delta < (1/p_A - 1/p_H)$ where Δ is the block propagation delay, p_A the probability that the adversary obtains a PoW and p_H the same probability for the honest parties. Given that $\Delta \geq dB/C$ where d is the diameter of the network and C the capacity of a channel between two nodes in the network in bytes per second, it follows that throughput is bounded by $(1/p_A - 1/p_H) \cdot C/d$.[28] It follows that (standard) Nakamoto consensus is suboptimal in terms of network utilization.

Sharding and layer-2 protocols. Blockchain sharding was introduced by Danezis and Meiklejohn [15] and Croman *et al.* [13]. In contrast to a fully replicated blockchain wherein each full node stores the complete ledger state, a sharded blockchain partitions the ledger state by assigning different committees who each maintain a different part of the ledger. Sharding could potentially enhance transaction throughput beyond the network's capacity by avoiding complete ledger replication across all parties. Instead, specific parties are tasked with maintaining shards, including distributing shard data to the public. Layer-2 (L2) protocols, see e.g., [29,40] offer a different approach to scaling: they typically centralize off-chain processing and then via incentives and/or zero-knowledge proofs ensure that the off-chain execution is correct. While both sharding and L2 approaches can be fundamental facilitators of further scaling – they address a different problem compared to the question of scaling a blockchain (L1) protocol to its absolute physical limits — which is the focus of our work.

On some tools used by Leios. Data-availability proofs based on Rabin's information dispersal [46] and the work by [9] have been widely used in the context of light clients or sharding (e.g., [52,1,35]). In contrast, Leios relies on signatures to ensure data availability since this is more amenable for our use in a gossip network.

Using pipelining to increase the throughput of BFT protocols has been proposed in [36] and was subsequently used in, e.g., [8,50]—which can be seen as an alternative way (to input endorsing) of decoupling payload dissemination from achieving consensus. Note that Leios pipelining is not directly related to that kind of pipelining since decoupling for the purpose of high throughput is already provided by input endorsing: in contrast, Leios pipelining is exclusively applied to provide timely-delivery guarantees for block-related data under freshest-first message delivery.

The idea of defending against message bursts by prioritizing fresher messages for download was introduced in [41] in form of downloading *towards* the freshest block in PoS Nakamoto consensus.

Equivocation proofs are (implicitly or explicitly) present in every Byzantine-Agreement/BFT protocol relying on digital signatures, dating all the way back to the initial work by Lamport *et al.* [34].

B Chernoff-Hoeffding bounds

We make use of the Chernoff-Hoeffding bound in form of Eq. (1.7) in [19].

Theorem 5 ([19]). *Let $X := \sum_{i\in[n]} X_i$ where X_i, $i \in [n]$ are independently distributed in $[0,1]$. Then, for $0 < \varepsilon < 1$,*

$$\Pr\left[X > (1+\varepsilon)E[X]\right] \leq \exp\left(-\frac{\varepsilon^2}{3}E[X]\right),$$

[28] Note that, in the worst case, each block producer includes transactions that are not yet contained in the other parties' mempools.

and

$$\Pr\left[X < (1-\varepsilon)E[X]\right] \leq \exp\left(-\frac{\varepsilon^2}{2}E[X]\right).$$

C Key Evolving Signatures

A *key evolving signature (KES)* is essentially a forward-secure digital signature scheme. As in classical signatures, a party can create a key pair that consists of a secret signing key and a public verification key. To allow for forward security the secret key is updated/evolved every time a signature is produced (for a bounded number of updates), while the old secret key is erased. This allows honest users to verify that a signature was generated at certain time period.

The above guarantees are abstracted by an idealized functionality $\mathcal{F}_{\mathsf{kes}}$ (cf. Figure 8). The main commands offered by $\mathcal{F}_{\mathsf{kes}}$ are (i) (Gen), answered by $(\text{Gen}, k_{\mathsf{kes}})$ for an (adversarially chosen) idealized public key k_{kes}, (ii) $(\text{Sign}, k_{\mathsf{kes}}, \mathbf{m}, j)$, answered by $(\text{Sign}, \boldsymbol{\sigma})$ where $\mathbf{m}$ and $\boldsymbol{\sigma}$ are vectors of messages and signatures respectively, and (iii) $(\text{Vfy}, k_{\mathsf{kes}}, j, m, \sigma)$, answered by $(\text{Vfy}, m, j, \sigma, k_{\mathsf{kes}}, f)$ with $f \in \{0,1\}$ indicating whether σ is a valid signature for message m at period j. For a realization of $\mathcal{F}_{\mathsf{kes}}$ from a key evolving signature scheme we point to [16].

Functionality $\mathcal{F}_{\mathsf{kes}}$

$\mathcal{F}_{\mathsf{kes}}$ is parameterized by the total number of updates T.

Variables: The functionality keeps track of the following variables, initialized to the values below:

1. $\mathsf{Keys} := \emptyset$: a set of tuples (P, v), where P is a party and v is a public key.
2. $\mathsf{Sigs} := \emptyset$: a set of tuples (m, j, σ, v, f) where m is a message, j is a period number, σ is a signature, v is a public key, and f is a bit.
3. A period counter $\mathsf{ctr}_P := 0$ for every party P.

Key generation: Upon receiving a message (Gen) from P, send (Gen, P) to the adversary. Upon receiving (Gen, P, v) from the adversary, verify that v does not appear in Keys or Sigs. Add (P, v) to Keys, set counter $\mathsf{ctr}_P := 1$, and pass (Gen, v) to P.

Sign and Update. Upon receiving a message $(\text{Sign}, v, j, \mathbf{m})$ from P:

1. If $(P, v) \notin \mathsf{Keys}$, or $j < \mathsf{ctr}_P$, or $j > T$, exit.
2. Send $(\text{Sign}, v, j, \mathbf{m})$ to the adversary and obtain a value $\boldsymbol{\sigma}$ such that no entry $(\mathbf{m}[i], j, \boldsymbol{\sigma}[i], v, 0)$ is recorded in Sigs, for all i. If it is, then output an error message to P and halt.
3. Add $(\mathbf{m}[i], j, \boldsymbol{\sigma}[i], 1)$ to Sigs for all i.
4. Set $\mathsf{ctr}_P := j + 1$.
5. Send $(\text{Sign}, \boldsymbol{\sigma})$ to P.

Signature Verification. Upon receiving a message $(\text{Vfy}, v', j, m, \sigma)$ from party P do:

1. If $v' = v$ and the entry $(m, j, \sigma, v, 1)$ is recorded in Sigs, then set $f := 1$. (This condition guarantees completeness: If the verification key v' is the registered one and σ is a legitimately generated signature for m, then the verification succeeds.)
2. Else, if $v' = v$, P is not corrupted, and no entry $(m, j, \sigma', v, 1)$ for any σ' is recorded in Sigs, then set $f := 0$ and record the entry $(m, j, \sigma, v, 0)$. (This condition guarantees unforgeability: If v' is the registered one, the signer is not corrupted, and never signed m, then the verification fails.)
3. Else, if there is an entry (m, j, σ, v', f') recorded in Keys, then let $f := f'$. (This condition guarantees consistency: All verification requests with identical parameters will result in the same answer.)
4. Else, if $j < \mathsf{ctr}_P$, let $f := 0$ and record the entry $(m, j, \sigma, v, 0)$. Otherwise, if $j = \mathsf{ctr}_P$, hand $(\text{Vfy}, m, j, \sigma, v')$ to the adversary. Upon receiving (Vfy, m, j, ϕ) from the adversary let $f := \phi$ and record the entry (m, j, σ, v', ϕ) in Sigs. (This condition guarantees that the adversary is only able to forge signatures under keys belonging to corrupted parties for time periods corresponding to the current or future slots.)

Output (Vfy, f) to P.

Fig. 8. Key-evolving signatures functionality $\mathcal{F}_{\mathsf{kes}}$.

D (Stake-Based) Voting

Leios requires a voting scheme that satisfies the following two conditions: (1) if all honest parties vote for a particular message, they are able to generate a certificate for that message; (2) if no honest party votes for a particular message, the adversary is unable to create such a certificate.

Protocol π_{sbv}

Variables: The protocol keeps track of the following variables, initialized to the values below:

1. $\mathsf{SD}^* := \bot$: stake distribution used.
2. $\mathsf{Keys} := \emptyset$: the set of public keys of the party.
3. A period counter ctr.

Key generation: Upon (GEN) from P: Pass (GETKEY) to $\mathcal{F}_{\mathsf{vrf}}$ and (GEN) to $\mathcal{F}_{\mathsf{kes}}$ and obtain keys k_{vrf} and k_{kes}, respectively. Add $(k_{\mathsf{vrf}}, k_{\mathsf{kes}})$ to Keys and pass (GEN, $(k_{\mathsf{vrf}}, k_{\mathsf{kes}})$) to P.

Stake distribution: Upon (SETSD, SD) from P: If $\mathsf{SD}^* = \bot$ and SD is valid, set $\mathsf{SD}^* := \mathsf{SD}$.

Voting: Upon (VOTE, $(k_{\mathsf{vrf}}, k_{\mathsf{kes}}), j, \mathbf{m}$) from P:

1. If $(k_{\mathsf{vrf}}, k_{\mathsf{kes}}) \notin \mathsf{Keys}$ or $j \leq \mathsf{ctr}$, exit.
2. Pass (EVAL, k_{vrf}, j) to $\mathcal{F}_{\mathsf{vrf}}$ and obtain (EVAL, y, π), where y is the VRF output and π is the VRF proof.
3. If $\mathsf{voteCount}(\mathsf{SD}^*, k_{\mathsf{vrf}}, y) = 0$, pass (VOTE, $\bot$) to P and exit.
4. Pass (SIGN, $k_{\mathsf{kes}}, j, \mathbf{m}'$) to $\mathcal{F}_{\mathsf{kes}}$, where $\mathbf{m}'[i] = (j, \mathbf{m}[i])$, and obtain (SIGN, $\mathbf{s}$), where $\mathbf{s}$ is a vector of signatures.
5. Let $\mathbf{v}$ be the vector of votes $\mathbf{v}[i] := (y, \pi, \mathbf{s}[i])$ for all i.
6. Set $\mathsf{ctr} := j$.
7. Pass (VOTE, $\mathbf{v}$) to P.

Vote verification: Upon (VFYVOTE, $(k_{\mathsf{vrf}}, k_{\mathsf{kes}}), j, m, (y, \pi, \sigma)$) from P:

1. Pass (VERIFY, $k_{\mathsf{vrf}}, j, y, \pi$) to $\mathcal{F}_{\mathsf{vrf}}$ and obtain (VERIFY, f_{vrf}), where f_{vrf} is a bit.
2. Set $f_{\mathsf{cnt}} := 1$ if $\mathsf{voteCount}(\mathsf{SD}^*, k_{\mathsf{vrf}}, y) > 0$ and $f_{\mathsf{cnt}} := 0$ otherwise.
3. Pass (VFY, $k_{\mathsf{kes}}, j, m, \sigma$) to $\mathcal{F}_{\mathsf{kes}}$ and obtain (VFY, f_{kes}), where f_{kes} is a bit.
4. Pass (VFYVOTE, $f_{\mathsf{vrf}} \wedge f_{\mathsf{cnt}} \wedge f_{\mathsf{kes}}$) to P.

Certificate generation: Upon (GENCERT, $\mathbf{k}, j, m, \mathbf{v}$) from P:

1. Run the verification in VFYVOTE with $(\mathbf{k}[i], j, m, \mathbf{v}[i])$ for all i; if not all of them succeed, exit.
2. If $\sum_i \mathsf{voteCount}(\mathsf{SD}^*, \mathbf{k}[i].k_{\mathsf{vrf}}, \mathbf{v}[i].y) < T$, exit.
3. Pass (GENCERT, $(\mathbf{k}, \mathbf{v})$) to P.

Certificate verification: Upon (VFYCERT, $j, m, (\mathbf{k}, \mathbf{v})$) from $\mathcal{F}_{\mathsf{sbv}}$:

1. Run the verification in VFYVOTE with $(\mathbf{k}[i], j, m, \mathbf{v}[i])$ for all i; set $f_{\mathsf{vfy}} := 1$ if all of them succeed and $f_{\mathsf{vfy}} := 0$ otherwise.
2. Set $f_{\mathsf{cnt}} := 1$ if $\sum_i \mathsf{voteCount}(\mathsf{SD}^*, \mathbf{k}[i].k_{\mathsf{vrf}}, \mathbf{v}[i].y) \geq T$ and $f_{\mathsf{cnt}} := 0$ otherwise.
3. Pass $f_{\mathsf{vfy}} \wedge f_{\mathsf{cnt}}$ to $\mathcal{F}_{\mathsf{sbv}}$.

Fig. 9. Protocol π_{sbv} to realize $\mathcal{F}_{\mathsf{sbv}}$.

These guarantees are captured by functionality $\mathcal{F}_{\mathsf{sbv}}$ (cf. Figure 2), which is realized by a protocol π_{sbv} via stake-based voting (SBV), relying on a verifiable random function (VRF) and a key-evolving signature scheme (KES). The protocol π_{sbv} is secure against an adaptive adversary corrupting less than half of the total stake.

D.1 The SBV Protocol

For completeness, this work provides a protocol π_{sbv} (cf. Figure 9) that realizes $\mathcal{F}_{\mathsf{sbv}}$. The protocol is based on sortition, i.e., in each period, a representative subset of the parties is elected in a VRF-based lottery and casts a vote. The certification generation is trivial, i.e., the votes are simply concatenated. This is sufficient for Leios, but more elaborate certification, in which votes are suitably aggregated, could be used to obtain more efficient realizations of $\mathcal{F}_{\mathsf{sbv}}$. The protocol is described in the $\{\mathcal{F}_{\mathsf{vrf}}, \mathcal{F}_{\mathsf{kes}}\}$-hybrid model.

Sortition. In the following it is assumed for simplicity that the stake distribution $\{s_P\}_P$ can be approximated by a set of weights w_P sucht that $s_P = w_P/W$ for $W = \sum_P w_P$. The sortition simply uses the output y of a VRF as randomness to sample a number of votes for party P from a binomial distribution $\mathrm{Bin}(w_P, p)$ for some parameter p used to calibrate the expected total number Wp of votes. For a stake distribution SD and the key k_{vrf} belonging to P, the number of votes is denoted by $\mathsf{voteCount}(\mathsf{SD}, k_{\mathsf{vrf}}, y)$.

Key generation. The public keys for π_{sbv} simply consist of public keys k_{vrf} and k_{kes} obtained from $\mathcal{F}_{\mathsf{vrf}}$ and $\mathcal{F}_{\mathsf{kes}}$, respectively.

Stake distribution. The party simply remembers the stake distribution input via command SETSD (if it is valid).

Vote generation and verification. To create votes for messages in a vector $\mathbf{m}$ in period j, a party first evaluates the VRF on j to obtain (y, π), where y is the VRF output and π is the proof. Then, it computes $\mathsf{voteCount}(\mathsf{SD}^*, k_{\mathsf{vrf}}, y)$. If the result is 0, the party was not elected to vote in period j and the output is $\bot$.

Protocol $\mathcal{S}_{\mathsf{sbv}}$

Variables: The simulator keeps track of the following variables, initialized to the values below:

1. $\mathsf{SD}^* := \perp$: stake distribution used.
2. $\mathsf{Keys} := \emptyset$: a set of tuples $(P, (k_{\mathsf{vrf}}, k_{\mathsf{kes}}))$, where P is a party and $(k_{\mathsf{vrf}}, k_{\mathsf{kes}})$ is a public key.

The simulator also internally runs copies of $\mathcal{F}_{\mathsf{vrf}}$ and $\mathcal{F}_{\mathsf{kes}}$ as well as of an adversary $\mathcal{A}$, connected to $\mathcal{F}_{\mathsf{vrf}}$ and $\mathcal{F}_{\mathsf{kes}}$.

Key generation: Upon $(\textsc{Gen}, P)$ from $\mathcal{F}_{\mathsf{sbv}}$: Pass, on behalf of P, $(\textsc{getKey})$ to $\mathcal{F}_{\mathsf{vrf}}$ and $(\textsc{Gen})$ to $\mathcal{F}_{\mathsf{kes}}$ and obtain keys k_{vrf} and k_{kes}, respectively. Add $(P, (k_{\mathsf{vrf}}, k_{\mathsf{kes}}))$ to Keys and pass $(k_{\mathsf{vrf}}, k_{\mathsf{kes}})$ to $\mathcal{F}_{\mathsf{sbv}}$.

Stake distribution: Upon $(\textsc{SetSD}, \mathsf{SD})$ from P: If $\mathsf{SD}^* = \perp$, set $\mathsf{SD}^* := \mathsf{SD}$. If $\mathsf{SD} \neq \mathsf{SD}^*$, simulate real world exactly for the remainder of the execution.

Voting: Upon $(\textsc{Vote}, P, (k_{\mathsf{vrf}}, k_{\mathsf{kes}}), j, \mathbf{m})$ from $\mathcal{F}_{\mathsf{sbv}}$:

1. Pass, on behalf of P, $(\textsc{Eval}, k_{\mathsf{vrf}}, j)$ to $\mathcal{F}_{\mathsf{vrf}}$ and obtain $(\textsc{Eval}, y, \pi)$, where y is the VRF output and π is the VRF proof.
2. If $\mathsf{voteCount}(\mathsf{SD}^*, k_{\mathsf{vrf}}, y) = 0$, pass $\perp$ to $\mathcal{F}_{\mathsf{sbv}}$ and exit.
3. Pass, on behalf of P, $(\textsc{Sign}, k_{\mathsf{kes}}, j, \mathbf{m}')$ to $\mathcal{F}_{\mathsf{kes}}$, where $\mathbf{m}'[i] = (j, \mathbf{m}[i])$, and obtain $(\textsc{Sign}, \mathbf{s})$, where $\mathbf{s}$ is a vector of signatures.
4. Let $\mathbf{v}$ be the vector of votes $\mathbf{v}[i] := (y, \pi, \mathbf{s}[i])$ for all i.
5. Pass $(\textsc{Vote}, \mathbf{v})$ to $\mathcal{F}_{\mathsf{sbv}}$.

Vote verification: Upon $(\textsc{VfyVote}, (k_{\mathsf{vrf}}, k_{\mathsf{kes}}), j, m, (y, \pi, \sigma))$ from $\mathcal{F}_{\mathsf{sbv}}$:

1. Pass $(\textsc{Verify}, k_{\mathsf{vrf}}, j, y, \pi)$ to $\mathcal{F}_{\mathsf{vrf}}$ and obtain $(\textsc{Verify}, f_{\mathsf{vrf}})$, where f_{vrf} is a bit.
2. Set $f_{\mathsf{cnt}} := 1$ if $\mathsf{voteCount}(\mathsf{SD}^*, k_{\mathsf{vrf}}, y) > 0$ and $f_{\mathsf{cnt}} := 0$ otherwise.
3. Pass $(\textsc{Vfy}, k_{\mathsf{kes}}, (j, m), \sigma, \pi)$ to $\mathcal{F}_{\mathsf{kes}}$ and obtain $(\textsc{Vfy}, f_{\mathsf{kes}})$, where f_{kes} is a bit.
4. Pass $f_{\mathsf{vrf}} \wedge f_{\mathsf{cnt}} \wedge f_{\mathsf{kes}}$ to $\mathcal{F}_{\mathsf{sbv}}$.

Certificate generation: Upon $(\textsc{GenCert}, \mathbf{k}, j, m, \mathbf{v})$ from $\mathcal{F}_{\mathsf{sbv}}$:

1. If $\sum_i \mathsf{voteCount}(\mathsf{SD}^*, \mathbf{k}[i].k_{\mathsf{vrf}}, \mathbf{v}[i].y) < T$, pass $\perp$ to $\mathcal{F}_{\mathsf{sbv}}$ and exit.
2. Pass $(\mathbf{k}, \mathbf{v})$ to $\mathcal{F}_{\mathsf{sbv}}$.

Certificate verification: Upon $(\textsc{VfyCert}, j, m, (\mathbf{k}, \mathbf{v}))$ from $\mathcal{F}_{\mathsf{sbv}}$:

1. Run the verification in $\textsc{VfyVote}$ with $(\mathbf{k}[i], j, m, \mathbf{v}[i])$ for all i; set $f_{\mathsf{vfy}} := 1$ if all of them succeed and $f_{\mathsf{vfy}} := 0$ otherwise.
2. Set $f_{\mathsf{cnt}} := 1$ if $\sum_i \mathsf{voteCount}(\mathsf{SD}^*, \mathbf{k}[i].k_{\mathsf{vrf}}, \mathbf{v}[i].y) \geq T$ and $f_{\mathsf{cnt}} := 0$ otherwise.
3. Pass $(\textsc{VfyVote}, f_{\mathsf{vfy}} \wedge f_{\mathsf{cnt}})$ to P.

Fig. 10. Simulator for protocol π_{sbv}.

Otherwise, the party signs $(j, \mathbf{m}[i])$ for all i with the KES scheme, obtaining a vector of signatures σ, and produces the votes $\mathbf{v}[i] = (y, \pi, \sigma[i])$.

Votes are verified by checking the VRF value y, the KES signature, and that $\mathsf{voteCount}$ on y is non-zero.

Certificate generation and verification. Certificates are generated by concatenation from any vector $\mathbf{v}$ of valid votes belonging to keys $\mathbf{k}$ with combined $\mathsf{voteCount}$ at least threshold T for the period in question. Given a fraction $\alpha < 1/2$ of relative adversarial stake, note that with high probability, the adversary will get a $\mathsf{voteCount}$ of approximately $\alpha W p$ whereas the honest parties will get approximately $(1 - \alpha)Wp$.

Certificate verification proceeds by verifying the individual votes and ensuring the $\mathsf{voteCount}$ is at least T.

D.2 Security

Environment. Protocol π_{sbv} securely realizes $\mathcal{F}_{\mathsf{sbv}}$ against an adaptive adversary corrupting at most half of the stake.

Theorem 6. *Let $0 \leq \alpha < 1/2$ be a constant and $0 < \delta < 1 - 2\alpha$. Further, let $W \in \mathbb{N}$ and $p \in (0, 1]$ as above. Then, π_{sbv} with threshold $T := (1 - \delta)(1 - \alpha)Wp$ securely UC-realizes $\mathcal{F}_{\mathsf{sbv}}$ against an adaptive adversary corrupting at most half of the stake, except with error $e^{-\Omega(\delta^2 \alpha W p)}$.*

Proof. The simulator for protocol $\mathcal{S}_{\mathsf{sbv}}$ is provided in Figure 10. It simulates keys, signatures, and certificates in a straightforward way.

To prove indistinguishability of the real and ideal worlds, it is only necessary to show that the "overrides" in $\mathcal{F}_{\mathsf{sbv}}$, where answers by the adversary are overwritten, only occur with small probability with $\mathcal{S}_{\mathsf{sbv}}$.

These overrides are:

1. Forcing the output of a vote when the adversary wants to output $\perp$ in $\textsc{Vote}$. This only happens if $\mathsf{voteCount}$ is 0 for all honest parties, which happens with probability at most $(1 - p)^{(1-\alpha)W}$.

2. Forcing the output of a certificate when the adversary wants to output $\perp$ in $\mathsf{GenCert}$. This only happens when the honest parties do not get a combined vote count of at least $(1-\delta)(1-\alpha)Wp$, which happens with probability at most $e^{-\Omega(\delta^2(1-\alpha)Wp)}$ by a simple Chernoff bound.
3. Forcing the output of $f = 0$ when the adversary has enough votes to cross the threshold $T = (1-\delta)(1-\alpha)Wp$ without honest votes. Using the fact that $\delta < 1-2\alpha$—and therefore $(1+\delta)\alpha < (1-\delta)(1-\alpha)$—this only happens with probability at most $e^{-\Omega(\delta^2\alpha Wp)}$ by a simple Chernoff bound.

The theorem follows. □

E Queue Clearing

Consider the following experiment with parameters β, τ, N, and p:

- There is an, initially empty, queue of messages, each with a timestamp in $\mathbb{N}$.
- At each time $t \in \mathbb{N}$:
  - $A_t \sim \mathrm{Bin}(N, p)$ new messages are generated, where the A_t are independent.
  - An adversarial scheduler determines at which point messages enter the queue.
  - Each message satisfying the following condition leaves the queue: there exists a $k \geq 0$ such that (i) the time the message spent in the queue is at least $\beta + k\tau$ and (ii) there are at most k newer messages in the queue.

A message is *timely*, if it is released into the queue at a time when its age was at most A_{max}. The goal in this section is to bound the expected time it takes for all *timely* messages present in the queue at some fixed time $\tilde{t}$ to clear the queue. This event is called *queue clearance w.r.t.* $\tilde{t}$.

Theorem 7. *Fix an arbitrary time $\tilde{t}$. Let T_{clear} be the time (post $\tilde{t}$) it takes for all* timely *messages in the queue at time $\tilde{t}$ to leave the queue. Assume that $\tau Np < 1$. Then, the expectation of T_{clear} is*

$$\mathbb{E}[T_{\mathsf{clear}}] \;\leq\; \frac{2\gamma^2}{(1-\tau Np)^2} + \frac{144\tau^2 Np^2}{(1-\tau Np)^4}\;,$$

where $\gamma := A_{\mathsf{max}} + \beta$.

Queue clearance in t steps after $\tilde{t}$. The first step is to bound the probability p_t that queue clearance does *not* occur up to $\tilde{t} + t$.

Lemma 9. *With parameters as above, assume that $\tau Np < 1$ and define*

$$\begin{aligned} t^* &:= \frac{\gamma}{1-\tau Np} \quad \textit{and} \\ \delta_0 &:= \frac{t-\gamma}{\tau t Np} - 1\,. \end{aligned}$$

Then for any $t \geq t^*$

$$p_t \;\leq\; \exp\left(-\frac{(t-1)Np\delta_0^2}{3}\right) \cdot \frac{3}{Np\delta_0^2}\,.$$

Observe that $\delta_0 = \delta_0(t)$ above depends on t. However, as shown below, for $t \geq 2t^*$,

$$\frac{Np\delta_0^2}{3} \;=\; \Theta\left(\frac{(1-\tau Np)^2}{\tau^2 Np}\right)$$

is independent of t.

Proof. First, observe that a message leaves the queue if, for some k, (1) the time it has spent in the queue (TSQ) is at least $\beta + k\tau$ and (2) there are at most k newer messages in the queue. Recall that for timely messages, the TSQ is lower bounded by the age of the message minus A_{max}.

Consider now the event that queue clearance has not occurred at time $\tilde{t} + t$. This implies that there is a message m for which

$$a + t - A_{\mathsf{max}} \;<\; \beta + K_{a,t} \cdot \tau \;,$$

where a is the age of m (at time $\tilde{t}$) and $K_{a,t} \sim \mathrm{Bin}((a+t)N, p)$ is the number of messages generated in the interval $[\tilde{t} - a, \tilde{t} + t]$ (including m). Hence, in said interval of length $a + t$, at least

$$K_{a,t} > (a + t - (A_{\mathsf{max}} + \beta))/\tau$$

messages must have been produced. Let E_a denote the event that there is such a message m of age a and define $p_t^{(a)} = \Pr[E_a]$. Then, by the union bound $p_t \leq \sum_{a=0}^{\tilde{t}} p_t^{(a)}$.

In preparation for applying a Chernoff bound to bound $p_t^{(a)}$, note that $\mathbb{E}[K_{a,t}] = (a+t)Np$ and that

$$p_t^{(a)} = \Pr[K_{a,t} > (1 + \delta_a)(a + t)Np]$$

where

$$\delta_a = \frac{a + t - \gamma}{\tau(a+t)Np} - 1 = \frac{1}{\tau Np} - \frac{\gamma}{\tau(a+t)Np} - 1 \tag{5}$$

using $\gamma := A_{\mathsf{max}} + \beta$. To simplify the analysis, it is convenient to use a fixed δ_0 for all $a \geq 0$. With this in mind, we observe from the last expression of (5) that δ_a is monotonically increasing in a and define δ_0 to be the value of δ_a arising at $a = 0$. Then, for any a

$$\delta_a \geq \delta_0 = \frac{t - \gamma}{\tau t Np} - 1 \,.$$

As the Chernoff bound requires $\delta_0 > 0$, this introduces the constraint

$$t \geq t^* = \frac{\gamma}{1 - \tau Np} \,.$$

For a particular a, applying a Chernoff bound we find that

$$p_t^{(a)} \leq \exp\left(-\frac{(t+a)Np\delta_0^2}{3}\right) = \exp\left(-\frac{(t-1)Np\delta_0^2}{3}\right) \cdot \exp\left(-\frac{(a+1)Np\delta_0^2}{3}\right)$$

and hence that

$$\begin{aligned}
p_t \leq \sum_{a=0}^{\tilde{t}} p_t^{(a)} &\leq \sum_{a=0}^{\tilde{t}} \exp\left(-\frac{(t-1)Np\delta_0^2}{3}\right) \cdot \exp\left(-\frac{(a+1)Np\delta_0^2}{3}\right) \\
&= \exp\left(-\frac{(t-1)Np\delta_0^2}{3}\right) \cdot \sum_{a \geq 0} \exp\left(-\frac{(a+1)Np\delta_0^2}{3}\right) \\
&\leq \exp\left(-\frac{(t-1)Np\delta_0^2}{3}\right) \cdot \int_0^{\infty} \exp\left(-\frac{aNp\delta_0^2}{3}\right) da \\
&= \exp\left(-\frac{(t-1)Np\delta_0^2}{3}\right) \cdot \frac{3}{Np\delta_0^2} \,,
\end{aligned}$$

as desired. □

Returning to $Np\delta_0^2/3$, one observes:

Lemma 10. *For $t \geq 2t^*$,*

$$F \;:=\; \frac{(1-\tau Np)^2}{12\tau^2 Np} \;\leq\; \frac{Np\delta_0^2}{3} \;\leq\; \frac{(1-\tau Np)^2}{3\tau^2 Np}\;.$$

Proof. First, recall that, by definition,

$$\delta_0 \;=\; \frac{t-\gamma}{\tau t Np} - 1 \;=\; \frac{t(1-\tau Np)-\gamma}{\tau t Np}\;.$$

Thus, $\delta_0 \leq (1-\tau Np)/(\tau Np)$. As for the lower bound, note that

$$t(1-\tau Np)-\gamma \;\geq\; \frac{1}{2}t(1-\tau Np)$$

under the assumption that $t \geq 2t^* = 2\gamma/(1-\tau Np)$, and, hence,

$$\delta_0 \;\geq\; \frac{1-\tau Np}{2\tau Np}\;.$$

The lemma follows. □

Computing the expectation. In order to compute the expectation, recall that Lemma 9 shows that $p_t \leq F^{-1}\cdot e^{-F(t-1)}$, for F determined by Lemma 10 and $t \geq 2t^*$. Then

$$\begin{aligned}
\mathbb{E}[T_{\mathsf{clear}}] \;&=\; \sum_{t=0}^{\infty} t\cdot \Pr[T_{\mathsf{clear}}=t] \;\leq\; \sum_{t=1}^{\infty}\Pr[T_{\mathsf{clear}}\geq t] \;\leq\; \sum_{t=1}^{\infty} p_{t-1} \;\leq\; \sum_{t=0}^{\infty} p_t \\
&\leq\; \sum_{t<2t^*} t + F^{-1}\sum_{t\geq 2t^*} e^{-F(t-1)} \\
&\leq\; 2(t^*)^2 + F^{-1}\int_{2t^*-1}^{\infty} e^{-F(t-1)}\,dt \\
&\leq\; 2(t^*)^2 + F^{-2}\,e^{-F(2t^*-2)} \\
&\leq\; 2(t^*)^2 + F^{-2} \\
&\leq\; \frac{2\gamma^2}{(1-\tau Np)^2} + \frac{144\tau^2 Np^2}{(1-\tau Np)^4}\;.
\end{aligned}$$

This establishes Theorem 7.

F Protocol Extensions

F.1 Multi-Epoch Protocol

To lift the overlay protocol to span multiple epochs (reflecting underlying stake change over the course of time), the respective mechanisms of the base protocol can be used (as for instance explicitly given in [16]). Once the underlying base protocol outputs the new stake distribution, that one is simply used for the IB/EB lottery and voting.

F.2 Different Consensus Resources

Our protocol extension is applicable to protocols based on other resources than stake, e.g., proof of work (PoW). Note, however, that the block/vote lotteries are tightly coupled to the pipeline stages—enabled by the fact that the VRF lotteries are tied to a particular time slot. To match these guarantees in a PoW

protocol, this would make it necessary for the PoW to additionally prove that it was not computed ahead of time. Such proofs can be given, e.g., by referencing a stable block in the base protocol (a mechanism used in [43])—but still giving the adversary a high degree of freedom of what "time stamps" to effectively choose (e.g., by referencing a newer, unstable block). To compensate this effect, the pipeline length L would have to grow to levels that would push liveness into intolerable regions.

This problem can be avoided by applying a mechanism in the spirit of [23]: input blocks, additionally to carrying transactions, also serve to measure each party's work contribution during the epoch—measured by their ratio of input blocks contributed. This measure is then transformed into a "virtual" stake distribution among the parties at the end of the epoch. Any PoS base protocol can then be applied while substituting stake by virtual stake.

Note that, as a consequence, the security of the underlying PoW protocol is somewhat degraded as public-key cryptography needs to be introduced, opening it up for new attack vectors (long-range attacks or the disability to recover from adversarial spikes).

F.3 Robustness under Extreme Conditions

Up to now we have analyzed the behavior of Leios under favorable network and adversarial conditions, i.e., the network does not experience any outages and honest parties control a majority of the stake. However, from a robustness perspective it is critical that the protocol remains secure under temporary disruptions such as network outages, even if these conditions are only occurring rarely. Otherwise, we risk the whole protocol collapsing from a single "black swan" event.

We examine here two such "bad" events of interest: temporary network outages, and temporary control of the relative (but not absolute) majority of the stake by the adversary. Given that for consensus Leios is dependent on the base-protocol, we cannot hope to get better guarantees that those provided by it. Hence, the main question which we explore in this section, is what security and performance characteristics Leios preserves when such events occur, assuming the base-protocol can tolerate them. Concretely, we are going to assume that (i) there is always enough bandwidth to safely run the base-protocol, and (ii) that the base-protocol self-heals, i.e., if agreement is disrupted due to a temporary dishonest relative majority, the system recovers after some time with parties again reaching a state of agreement. [29]

Assume now that both "bad" events occur. First off, given that no *absolute* stake majority is formed, the worst thing that can happen in the Leios layer is either no EB being certified or an EB containing only adversarial IBs being certified. In any case, no purely adversarial and possibly incorrect EB certificates are produced, and thus *IB availability* is always ensured albeit with some extra delays in delivery due to the temporary network outages. Now, our effort earlier in decoupling consensus from block availability pays out as only IBs for which delivery is guaranteed are serialized by the base-protocol. Hence, IB delivery or EB certification failures do not affect consistency. Similarly, in the case where the consistency of the base-protocol is temporarily disrupted, the recovery of the base-protocol directly implies the recovery of Leios. Summarizing, Leios inherits the self-healing property of the base-protocol, while throughput and liveness may degrade temporarily when such disruptions occur.

Finally, due to the modular design of Leios, and the fact that only a single EB certificate needs to be included in each base-protocol block, throughput and liveness at a similar level as that of the base-protocol can be guaranteed under such extreme conditions. Specifically, the base-protocol can serve a *dual* role by containing both an EB certificate and regular transactions. Now, even if no honest IB makes it to a certified EB or no certified EB is produced at all, some of the throughput is still retained intact due to the regular transactions being included in the base-protocol blocks.

[29] In fact, both PoW and PoS low-throughput blockchain protocols have been shown to provably satisfy this property [2,4], starting with Bitcoin.